



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Facultat d'Informàtica de Barcelona



IMPLEMENTACIÓ D'UN INTÈRPRET PAS A PAS PER A LA PROGRAMACIÓ FUNCIONAL

GENÍS CARRETERO FERRETE

Director/a

GERARD ESCUDERO BAKX (Departament de Ciències de la Computació)

Titulació

Grau en Enginyeria Informàtica (Computació)

Memòria del treball de fi de grau

Facultat d'Informàtica de Barcelona (FIB)

Universitat Politècnica de Catalunya (UPC) - BarcelonaTech

01/07/2026

Índex

I	Gestió del Projecte (GEP)	7
0.1	Introducció	8
0.2	Contextualització del Projecte	8
0.2.1	Marc del projecte i contextualització a la FIB	8
0.2.2	Definició de termes i conceptes clau	9
0.2.3	Identificació del problema a resoldre	10
0.2.4	Actors implicats i beneficiaris	10
0.2.5	Estat de l'art i alternatives existents	11
0.2.6	Justificació del projecte i valor afegit	12
0.3	Abast i Objectius del Projecte	13
0.3.1	Objectiu General	14
0.3.2	Objectius Específics	14
0.3.3	Límits del sistema i elements fora de l'abast	14
0.3.4	Requeriments Tècnics i Funcionals	15
0.3.5	Identificació de Riscos i Plans de Mitigació	16
0.3.6	Lliurables del Projecte	17
0.4	Metodologia i Rigor de Desenvolupament	17
0.5	Planificació Temporal	19
0.5.1	Gestió del Calendari i Distribució d'Hores	19
0.5.2	Descripció Detallada de les Tasques, Precedències i Rols	19
0.5.3	Recursos Humans i Materials	20
0.5.4	Diagrama de Gantt	20
0.6	Pressupost i Sostenibilitat	22
0.6.1	Identificació i Justificació de Costos	22
0.6.2	Control de Gestió de Desviacions	23
0.6.3	Informe de Sostenibilitat	24
II	Desenvolupament Tècnic	25
1	Anàlisi Lèxica i Sintàctica	26
1.1	Disseny de la Gramàtica i l'Arbre de Sintaxi Abstracta	27

1.1.1	Suficiència Teòrica del Nucli d'Expressions	27
1.1.2	El Tractament de la Recursivitat mitjançant Supercombinadors	28
1.2	L'Abstracció de Declaracions (<i>Bindings</i>)	28
1.3	Implementació i Arquitectura del Parser Monàdic	29
1.3.1	Fonamentació Teòrica i la Mònada Parser	29
1.3.2	Els Maons Sintàctics: Parsers Monàdics Primitius	30
1.3.3	Gestió de l'Entorn i Aïllament de l'Espai en Blanc: El combinador <code>token</code>	31
1.3.4	Gestió de Precedència i Descens Sintàctic Jerarquitzat	31
1.3.5	Aplicació de Funcions i Currificació Sintàctica	32
1.4	Arbre de Sintaxi Abstracta Resultant	33
2	Sistema de Tipus	35
2.1	Fonaments de l'Algorisme de Hindley-Milner	35
2.2	L'Algorisme d'Unificació de Robinson i el Test d'Ocurrencia	36
2.3	El Motor d'Inferència Estructural	37
2.3.1	Polimorfisme Paramètric i Instanciació de Variables Fresques	38
2.3.2	Anàlisi de Casos Claus de la funció infereix	39
2.4	Representació i Estètica Formats (Pretty-Printing)	42
3	Avaluació i Reducció de Grafs	43
3.1	De l'AST al Graf	43
3.1.1	Estructura del Heap i Adreçament	44
3.2	Lambda Lifting i Supercombinadors	45
3.2.1	Extracció de variables lliures	46
3.2.2	Generació de funcions globals	46
3.3	Estratègia de Reducció (Lazy Evaluation)	47
3.3.1	Exploració del Camí Esquerre (Spine Unwinding)	48
3.3.2	Cas Pràctic: Avaluació Pas a Pas de l'Expressió Guia	49
4	Resultats i Proves	51
4.1	Bateria de Tests d'Inferència	51
4.1.1	Verificació de l'AST (<code>test_ast.hs</code>)	51
4.1.2	Inferència Exitosa i Polimorfisme (<code>test_type.hs</code>)	52
4.1.3	Tests Negatius i Robustesa	52
4.2	Execució de Supercombinadors	52
4.3	Limitacions actuals	53
5	Conclusions	54
5.1	Objectius assolits	54

5.2	Aprenentatges personals	55
5.3	Treball futur	55
	Bibliografia	57

Abstract

Aquest Treball de Final de Grau presenta el disseny i la implementació d'un interpret educatiu pas a pas per a un subconjunt del llenguatge de programació funcional Haskell, desenvolupat íntegrament en Haskell. El projecte neix amb l'objectiu de resoldre la desconexió cognitiva que pateixen els estudiants en introduir-se al paradigma funcional pur, on l'avaluació mandrosa (lazy evaluation) i les opacitats dels compiladors tradicionals, com GHCi, dificulten l'aprenentatge. L'arquitectura del sistema es divideix en tres grans mòduls: un analitzador lèxic i sintàctic construït des de zero mitjançant combinadors de parsers monàdics, evitant l'ús de generadors de codi externs; un motor d'inferència de tipus estàtic basat en l'algorisme de Hindley-Milner i la unificació de Robinson; i un avaluador mandrós fonamentat en la reducció de grafs i l'ús de supercombinadors. Els resultats demostren la viabilitat d'una implementació modular i declarativa capaç de processar estructures complexes com l'aplicació parcial, la currificació i les clausures lèxiques. Aquesta eina assoleix una fita tècnica de gran rigor i esdevé un recurs pedagògic clau per fer transparent el procés d'execució i facilitar la comprensió de les bases teòriques de la programació funcional.

This Bachelor's Thesis presents the design and implementation of an educational, step-by-step interpreter for a subset of the functional programming language Haskell, developed entirely in Haskell. The project stems from the objective of bridging the cognitive gap students face when introduced to the pure functional paradigm, where lazy evaluation and the black-box nature of traditional compilers, such as GHCi, hinder the learning process. The system's architecture is divided into three main modules: a lexical and syntactic analyzer built from scratch using monadic parser combinators, without relying on external code generators; a static type inference engine based on the Hindley-Milner algorithm and Robinson's unification; and a lazy evaluator founded on graph reduction and supercombinators. The results demonstrate the viability of a modular and declarative implementation capable of processing complex structures such as partial application, currying, and lexical closures. This tool achieves a highly rigorous technical milestone and serves as a key pedagogical resource to make the execution process transparent and facilitate the understanding of the theoretical foundations of functional programming.

Este Trabajo de Final de Grado presenta el diseño y la implementación de un intérprete educativo paso a paso para un subconjunto del lenguaje de programación funcional Haskell, desarrollado íntegramente en Haskell. El proyecto nace con el objetivo de resolver la desconexión cognitiva que sufren los estudiantes al introducirse en el paradigma funcional puro, donde la evaluación perezosa (lazy evaluation) y la opacidad de los compiladores tradicionales, como GHCi, dificultan el aprendizaje. La arquitectura del sistema se divide en tres grandes módulos: un analizador léxico y sintáctico construido desde cero mediante

combinadores de parsers monádicos, sin dependencias de generadores de código externos; un motor de inferencia de tipos estático basado en el algoritmo de Hindley-Milner y la unificación de Robinson; y un evaluador perezoso fundamentado en la reducción de grafos y el uso de supercombinadores. Los resultados demuestran la viabilidad de una implementación modular y declarativa capaz de procesar estructuras complejas como la aplicación parcial, la currificación y las clausuras léxicas. Esta herramienta alcanza un hito técnico de gran rigor y se convierte en un recurso pedagógico clave para hacer transparente el proceso de ejecución y facilitar la comprensión de las bases teóricas de la programación funcional.

Part I

Gestió del Projecte (GEP)

0.1 Introducció

El present Projecte de Final de Grau (TFG) consisteix en el disseny, anàlisi i implementació d'un intèrpret educatiu per a un subconjunt del llenguatge de programació funcional Haskell. Aquesta eina neix amb el propòsit fonamental de resoldre la desconexió existent entre el model mental de l'alumne que s'inicia en el paradigma funcional i l'execució real que efectua la màquina.

A diferència dels llenguatges imperatius on l'estat de les variables i el flux de control s'alteren pas a pas, els llenguatges funcionals com Haskell operen mitjançant l'avaluació d'expressions de manera mandrosa (*lazy evaluation*). L'objectiu principal d'aquest projecte és obrir la "caixa negra" de l'avaluador funcional, creant una eina que permeti visualitzar l'arbre sintàctic abstracte i el procés de reducció d'expressions matemàtiques de forma interactiva, facilitant així la comprensió de conceptes complexos com la recursivitat pura, l'aplicació parcial de funcions i el filtratge per patrons (*pattern matching*).

0.2 Contextualització del Projecte

0.2.1 Marc del projecte i contextualització a la FIB

El present TFG s'emmarca dins de la intensificació en Computació del Grau en Enginyeria Informàtica impartit a la Facultat d'Informàtica de Barcelona (FIB) de la Universitat Politècnica de Catalunya (UPC). Concretament, el projecte està concebut per integrar-se de manera directa en les eines de suport docent de l'assignatura de Llenguatges de Programació (LP), una assignatura nuclear i obligatòria per a tots els estudiants del grau.

El pla d'estudis de la FIB està estructurat de manera que l'estudiant adquireix en els seus primers anys una formació molt sòlida en els paradigmes imperatiu i orientat a objectes, utilitzant llenguatges com C++ (a PRO1 i PRO2) i Java. Aquests paradigmes es basen en la mutabilitat de l'estat, els bucles estructurats (for, while) i una execució estrictament seqüencial. No obstant això, a l'assignatura de LP, els alumnes s'enfronten per primera vegada al paradigma funcional pur mitjançant el llenguatge Haskell.

Aquest salt cognitiu representa, històricament, una de les majors dificultats acadèmiques del grau. La programació funcional pura exigeix pensar en termes de transformació de dades i composició matemàtica en lloc d'instruccions de modificació de memòria. La natu-

ralesa mandrosa de Haskell agreuja aquesta transició, ja que l'ordre en què s'escriu el codi no correspon necessàriament a l'ordre en què la màquina l'avalua. Aquest TFG pretén proporcionar una eina docent directament aplicable als laboratoris de la FIB, alineant-se amb la competència transversal d'aprenentatge autònom i millorant significativament la ràtio de comprensió de la semàntica del llenguatge.

0.2.2 Definició de termes i conceptes clau

Per entendre l'abast teòric i la complexitat arquitectònica del projecte, és imprescindible definir amb rigor una sèrie de conceptes fonamentals propis de la teoria de compiladors i del paradigma funcional, que seran el nucli del motor a desenvolupar:

- **Programació Funcional Pura:** Un paradigma de programació declaratiu on els programes es construeixen exclusivament aplicant i component funcions matemàtiques pures. En aquest model no existeixen els efectes secundaris (*side-effects*) ni la mutabilitat de variables. Una funció, donats els mateixos arguments, retornarà sempre el mateix resultat.
- **Avaluació Mandrosa (Lazy Evaluation):** És una estratègia d'avaluació de computació que retarda el càlcul d'una expressió fins al moment en què el seu valor és estrictament necessari per a obtenir el resultat final del programa. Permet la creació i manipulació d'estructures de dades infinites i evita càlculs innecessaris.
- **Thunk:** Un encapsulament (sovint implementat com un punter a una funció i el seu entorn) que s'utilitza en l'avaluació mandrosa per representar una expressió no avaluada. L'interpret "força" *think* quan realment en necessita el valor.
- **Reducció de Grafs (Graph Reduction):** Mecanisme intern mitjançant el qual s'avaluen les expressions en Haskell. Les expressions es modelen com un graf dirigit acíclic en memòria. Els passos d'execució consisteixen a localitzar un subgraf avaluable (anomenat *redex*) i aplicar regles de reescriptura (beta-reduccions) fins a obtenir una forma normal.
- **Aplicació parcial (Currying):** Procés teòric pel qual una funció que rep múltiples arguments es transforma en una seqüència de funcions, cadascuna amb un sol argument. És la base del sistema de tipus funcional.
- **Filtratge per patrons (Pattern Matching):** Un mecanisme molt potent de control de flux i desestructuració de dades. Permet comprovar si una estructura complexa té una forma específica i enllaçar els seus components a variables d'àmbit local per al seu tractament.

- **Arbre Sintàctic Abstracte (AST - Abstract Syntax Tree):** Una representació en forma d'arbre de l'estructura lògica del codi font. Cada node de l'arbre denota un constructe del codi (una operació, una declaració, etc.). El parser serà l'encarregat de generar aquest AST a partir del text escrit per l'alumne.
- **Monadic Parser Combinators:** Una tècnica avançada de programació funcional per a l'anàlisi lèxica i sintàctica. Permet tractar els *parsers* (analitzadors) com a valors de primera classe. Mitjançant operacions monàdiques, es poden combinar petits analitzadors (ex. un que llegeix un dígit) per construir analitzadors de gramàtiques altament complexes [3].

0.2.3 Identificació del problema a resoldre

El problema troncal que motiva aquest TFG és la manca de visibilitat i retroalimentació formativa en el procés d'execució del codi funcional per a estudiants primerencs. Actualment, el procés d'aprenentatge es dona de la següent manera: un estudiant implementa un algorisme recursiu en l'interpret oficial (el Glasgow Haskell Compiler o GHCi), l'executa, i el sistema li retorna únicament el valor final. Si el resultat és erroni, o si s'esdevé un error com un bucle infinit o un *Stack Overflow*, l'interpret actua com una "caixa negra".

Aquesta opacitat significa que l'alumne no rep informació sobre *quina* substitució concreta ha provocat l'error o *com* ha evolucionat l'acumulador en una funció d'ordre superior com un `foldl` o `foldr`. La frustració derivada d'aquest fet allarga innecessàriament les hores de laboratori. La incapacitat d'observar com creix una expressió abans de ser reduïda fa que molts alumnes memoritzin patrons de codi en comptes d'interioritzar la semàntica del model de grafs. El problema a resoldre, per tant, és la manca d'un entorn pedagògic visual que "alenteixi la màquina i forci la transparència pas a pas, prioritant la claredat educativa envers el rendiment d'execució.

0.2.4 Actors implicats i beneficiaris

S'han identificat i caracteritzat els següents perfils relacionats directament o indirectament amb el cicle de vida del projecte:

1. **Estudiants universitaris (Beneficiaris directes):** Els alumnes de l'assignatura LP de la FIB i qualsevol persona que s'estigui introduint en Haskell. Són els usuaris finals de l'aplicació. Utilitzaran l'interpret per verificar hipòtesis de funcio-

nament, entendre els errors del seu codi i assimilar conceptes abstractes observant gràficament les transformacions del codi pas a pas.

2. **Equip Docent i Professorat (Facilitadors i beneficiaris indirectes):** Els professors de laboratori i de teoria de la FIB. El professorat disposarà d'una eina per il·lustrar de manera dinàmica exemples complexos a la pissarra o projector. Podran crear exercicis on els estudiants hagin de predir el "següent pas" de reducció abans de fer clic al botó de l'interpret.
3. **Institució (La Facultat i el Departament de CS):** La FIB es beneficia en millorar les seves ràtios d'aprovats i de satisfacció a les assignatures de programació avançada, dotant-se d'eines docents de creació pròpia.
4. **Desenvolupador i Investigador (Jo):** Com a autor d'aquest TFG, actuo com a analista de requeriments, arquitecte de programari i desenvolupador, aprofundint els coneixements en la construcció de compiladors, l'enginyeria de programari àgil i el disseny de la interacció usuari.

0.2.5 Estat de l'art i alternatives existents

L'ecosistema d'eines al voltant del desenvolupament per a Haskell és profund i madur, però està fortament orientat a l'entorn industrial i a enginyers de programari sèniors. Una anàlisi de les eines existents permet evidenciar per què s'ha de dissenyar una solució completament nova des de zero per a l'àmbit docent.

- **GHCi Debugger (oficial):** És el depurador inclòs de sèrie amb GHC. Tot i permetre introduir punts d'aturada (*breakpoints*) i inspeccionar variables, el seu ús és exclusivament per línia de comandes textual. El seu gran defecte educatiu és que la informació que bolca correspon a l'estat intern del compilador (conegut com a Haskell Core), on el codi ja ha patit nombroses optimitzacions i *desugarings* (eliminació de "sucre sintàctic"), sent sovint incompreensible respecte al codi original que l'alumne havia escrit.
- **Eines acadèmiques de visualització de grafs (GHood i Haskell Visualizer):** Durant els anys 2000 es van publicar algunes tesis i aplicacions (com GHood) per visualitzar el graf d'avaluació de Haskell en programes amb Java. El problema és que estan basades en versions antiquades de la sintaxi, requereixen processos complexos de compilació (no són **plug-and-play**), i només mostren una topologia d'arbre estàtica un cop el programa ja ha finalitzat, en lloc de permetre a l'alumne prémer "següent" per veure el canvi immediat a la línia de codi.

- **Hoed:** Una eina moderna de depuració algorísmica per a Haskell. És molt potent però segueix enfocada a usuaris avançats. Utilitza tècniques de instrumentació de codi invasives (l'usuari ha de posar etiquetes `observe` per tot el codi font per poder veure el seu valor). Això requereix aprendre una eina per poder aprendre el llenguatge, afegint fricció.
- **Python Tutor (L'Estàndard Docent de Referència):** Aquesta eina web creada per Philip Guo és el referent mundial absolut en docència de programació. Permet veure el codi Python, Java o C a l'esquerra i la pila de crides i punters a la dreta pas a pas. No obstant això, Python Tutor no suporta cap llenguatge funcional pur. La seva arquitectura està dissenyada per a l'avaluació estricta i els mapes de memòria imperatius, la qual cosa fa impossible adaptar-la directament a un sistema de reducció de grafs d'avaluació mandrosa.

L'absència d'un "Haskell Tutor" equivalent és el buit exacte que aquest TFG pretén omplir.

0.2.6 Justificació del projecte i valor afegit

Per tal de justificar de manera diàfana i esquemàtica el valor d'aquesta solució envers el que ja ofereix el mercat, es presenta la següent taula comparativa detallada:

Taula 1: Comparativa de la solució proposada respecte a les alternatives actuals

Característica	Avaluada	El Nostre Intèrpret (TFG)	GHCi	Py. Tutor
Públic objectiu principal		Estudiants i professors (Àmbit Acadèmic)	Enginyers/Sèniors	Principiants (Acadèmic)
Llenguatge suportat		Subconjunt docent de Haskell (FIB)	Haskell complet	Python/C/Java
Visualització gràfica del procés d'avaluació		Sí (Text ressaltat pas a pas i AST)	No (Consola textual incomprensible)	Sí (Variables en memòria)
Control temporal de l'execució		Navegació interactiva endavant i endarrere	Només salt endavant entre punts d'aturada	Endavant i endarrere
Requisits d'Instal·lació		Cap (Accés via web o executable portable)	Alta (Requereix GHC, Cabal o Stack)	Cap (Navegador)
Tractament explícit de l'avaluació mandrosa		Representació gràfica de <i>Thunks</i> pendents	Amagat internament	N/A (És estricte)

A més de l'evident salt en usabilitat que s'observa a la taula, a nivell arquitectònic l'aportació clau és la utilització del paradigma de **Monadic Parser Combinators** [4, 6] per al desenvolupament del nucli (*core*). Aquesta metodologia, en detriment d'usar eines automàtiques de generació de parsers com Lex/Yacc o Alex/Happy, proporciona dos beneficis fonamentals: primer, el codi de l'interpret s'escriu completament en Haskell natiu; i segon, la definició del parser s'assembla pràcticament línia a línia a la gramàtica BNF oficial. Això és decisiu perquè qualsevol professor de la FIB, en el futur, pugui obrir el codi d'aquest TFG i afegir una nova funcionalitat sintàctica sense haver de ser un expert en eines de tercers. L'arquitectura garantirà, a més, la capacitat de generar missatges d'error personalitzats fets a mida per l'assignatura.

0.3 Abast i Objectius del Projecte

L'establiment rigorós de l'abast és vital per no incorrer en una expansió incontrolada de les prestacions del programari (*scope creep*), donada la immensitat del llenguatge Haskell.

0.3.1 Objectiu General

L'objectiu central i global d'aquest projecte és **dissenyar, programar, testar i validar** un intèrpret educatiu funcional per a un subconjunt controlat del llenguatge Haskell, dotat amb una interfície d'usuari (UI) interactiva que exposi al detall cada pas computacional de la semàntica de reducció, convertint l'execució en un procés transparent i formatiu.

0.3.2 Objectius Específics

Per assolir amb èxit l'objectiu general, s'estableixen els següents sub-objectius seqüencials:

- **Definir** formalment els límits del subconjunt del llenguatge Haskell que l'eina serà capaç de suportar. Això inclourà tipus bàsics (enters, booleans), llistes simples, definicions de funcions mitjançant equacions, estructures condicionals (**if-then-else**), aplicació parcial i funcions d'ordre superior clàssiques (**map**, **filter**, **fold**).
- **Construir** l'analitzador lèxic (que tokenitzi l'entrada) i l'analitzador sintàctic emprant la llibreria *Parsec* o *Megaparsec*, assegurant una alta tolerància i amabilitat en la reportació d'errors lèxics de l'usuari.
- **Implementar** el motor de reducció basat en un entorn (*Environment*), dissenyant un sistema d'avaluació estructurada que pugui interrompre l'execució a cada aplicació delta o beta, guardar-ne l'estat en memòria i continuar sota demanda seguint la regla "lazy".
- **Desenvolupar** una Interfície Gràfica d'Usuari (GUI) intuïtiva, minimalista i robusta on els estudiants puguin enganxar el seu codi al panell esquerre i navegar per una traça temporal visual d'execució al panell dret.
- **Validar** el funcionament correcte i la idoneïtat pedagògica del conjunt de l'eina, realitzant proves sobre bateries de problemes reals trets directament dels exàmens i pràctiques històriques de l'assignatura de LP de la FIB.

0.3.3 Límits del sistema i elements fora de l'abast

Tan important com saber què s'ha de fer és tenir clar allò que **no** està inclòs per mantenir el projecte dins els marges de viabilitat de temps (540 hores). Queden exclosos explícitament els següents aspectes de Haskell:

- El sistema complet d'inferència de tipus (tipus Hindley-Milner). L'interpret assumirà que el codi introduït és semànticament correcte i tipat fort, confiant en la reducció mecànica. No es desenvoluparà un *Type Checker* estricte prèvi.
- Operacions d'Entrada/Sortida (I/O). No s'implementarà la mònada IO ni funcions com `putStrLn` o `readFile`, atès que el projecte se centra en expressions pures matemàtiques.
- Suport avançat a les Classes de Tipus (*Typeclasses*) i instanciació de variables complexes de context.
- Mòduls, importacions d'arxius externs o el paquet base de biblioteques estàndard (a excepció de les operacions aritmètiques primitives pre-carregades a l'entorn).

0.3.4 Requeriments Tècnics i Funcionals

Dels objectius anteriors es destil·len els requeriments del producte de programari:

Requeriments Funcionals (RF):

- **RF1 (Entrada):** L'aplicació ha de disposar d'un panell d'edició on l'usuari pugui introduir el codi font en format de text lliure, amb ressaltat de sintaxi Haskell.
- **RF2 (Parsing i Gestió d'Errors):** En sol·licitar l'execució, l'interpret ha de transformar el text en un AST i aturar-se de forma immediata mostrant la línia i columna exacta on l'usuari hagi comès un error sintàctic.
- **RF3 (Motor de Traça):** El sistema ha de calcular i retenir en estructures de dades seqüencials tots els estats intermedis del codi, emulant els passos lògics d'un ésser humà reduint amb llapis i paper.
- **RF4 (Control):** S'ha d'oferir a l'usuari una botonera estil reproductor multimèdia: Reproduir cap al final, Següent Pas, Pas Anterior, i Tornar a l'inici.
- **RF5 (Ressaltat de Redex):** A cada instant de la traça visual, l'aplicació ha de marcar (per exemple amb un color diferent o en negreta) quina sub-expressió exacta s'acaba d'avaluar o quina s'avaluarà immediatament després (el *redex*).

Requeriments No Funcionals (RNF):

- **RNF1 (Usabilitat - Zero Fricció):** Seguint les heurístiques de Nielsen, l'eina ha de garantir que un usuari assoleixi executar el seu primer codi amb un màxim de

dos clics des de l'inici de la sessió. Ha de requerir zero descàrregues o instal·lacions (prioritzant la tecnologia d'entorn d'aplicació web SPA).

- **RNF2 (Eficiència Temporal):** Tot i no estar optimitzat per a grans processos corporatius, l'anàlisi previ del codi docent no ha de superar un llindar màxim d'espera de 3 segons per evitar pèrdua d'atenció per part de l'estudiant.
- **RNF3 (Mantenibilitat):** El codi font estarà ben documentat (usant *Haddock* o documentació en línia equivalent) facilitant-ne futures extensions al departament de CS de la FIB.

0.3.5 Identificació de Riscos i Plans de Mitigació

L'enginyeria d'un compilador docent comporta un seguit de riscos tècnics i conceptuals d'alt impacte que requereixen d'estratègies preventives adequades:

- **Risc 1: Explosió combinatòria de la memòria per recursivitat infinita.**
 - *Explicació:* Estudiants introduint bucles infinits sense condició de parada col·lapsaran la memòria RAM de l'eina intentant guardar els passos històrics.
 - *Mitigació (Pla B):* Establir per defecte un tall de seguretat forçat (per exemple, aturar la reducció asíncrona si s'arriba a 1.000 passos) avisant l'usuari amb un missatge clar de "Possibilitat de recursivitat infinita detectada".
- **Risc 2: Dificultat i lentitud en la instrumentació de l'avaluació mandrosa.**
 - *Explicació:* Gestionar estats immutables i clausures (*closures*) mentre s'extreu gràficament la informació pot presentar una dificultat superior al model imperatiu inicialment previst.
 - *Mitigació (Pla B):* Si el disseny del motor de grafs resulta inabastable als dos mesos de desenvolupament, es reduirà l'abast del projecte per suportar inicialment només l'avaluació aplicativa (estRICTA) limitant funcions infinites, però assegurant en qualsevol cas el lliurament d'un intèrpret funcional i educatiu.
- **Risc 3: Ambigüitats en l'anàlisi gramatical (Parsing).**
 - *Explicació:* La gramàtica de Haskell és famosa per algunes particularitats estructurals com la regla d'identació significativa (*off-side rule*) o la complexitat de l'aplicació infixa d'operadors.

- *Mitigació (Pla B)*: Eliminar l'anàlisi basada en identació i requerir de manera obligatòria claus explícites i punts i coma { ; } per a la versió inicial de l'interpret docent, fet que simplifica enormement l'AST generat.

0.3.6 Lliurables del Projecte

El lliurable principal d'aquest Treball de Final de Grau és la implementació completa de l'interpret funcional. El codi font, estructurat de forma modular i escrit íntegrament en Haskell, juntament amb la bateria de tests, és de codi obert i es troba allotjat de manera pública i accessible al següent repositori oficial: <https://github.com/geniscarretero/interpret-pas-a-pas-prog-funcional>

0.4 Metodologia i Rigor de Desenvolupament

L'elevada incertesa de dissenyar una aplicació altament tècnica fa inviable l'ús de metodologies clàssiques o predictives (model en Cascada). En conseqüència, es seguirà una **metodologia iterativa i incremental** amb un fort enfocament àgil basat en les cerimònies i pràctiques del **marc de treball Scrum** [2]. Malgrat ser un projecte individual, l'alumne aplicarà aquest rigor assumint de facto els rols fonamentals: actuarà com a *Product Owner* definint els requeriments i alhora com a equip de desenvolupament de manera autoorganitzada.

Estructura de Cicles Iteratius (Sprints): El projecte s'estructurarà en cicles anomenats *Sprints* d'una durada constant de 2 a 3 setmanes. Al finalitzar cadascun d'aquests Sprints s'assolirà una fita rellevant; la generació d'un Increment del Producte potencialment entregable. La lògica evolutiva de l'increment serà:

- *Sprint 1*: Analitzador que suporta operacions enteres i literals.
- *Sprint 2*: Suport a declaració i aplicació de funcions simples.
- *Sprint 3*: Gestió i expansió de llistes i *Pattern Matching*.
- *Sprint 4*: Motor iteratiu complet amb historial de traça.
- *Sprints 5-6*: Frontend gràfic, resolució de deute tècnic i optimització.

Aquest model basat en el concepte de Producte Mínim Viable (MVP) contínu permet

adaptar la direcció tècnica o retallar funcionalitats perifèriques depenent del temps real dedicat, sense comprometre en cap cas la viabilitat nuclear de l'entrega final.

Eines, Protocols i Control Qualitatiu:

- **Gestió del Flux (Tauler Kanban):** S'emprarà un tauler a Jira o Trello per centralitzar l'estat del projecte. Les funcions principals es redactaran com a Històries d'Usuari i es traslladaran successivament per les columnes *Product Backlog*, *To Do*, *In Progress*, *Testing*, i *Done*.
- **Control de Versions Sòlid (Git i GitHub):** Tot el desenvolupament de codi es traçarà amb el sistema Git de repositoris històrics. S'utilitzarà el protocol *Feature Branch Workflow*, on cada nova funcionalitat es codificarà en una branca independent abans de ser fusionada a la principal mitjançant *Pull Requests*. Això és una garantia d'integritat del codi.
- **Polítiques de Validació Contínua (TDD i BDD):** Com a garantia de fiabilitat del core computacional del compilador, la pràctica de proves s'executarà simultàniament a la creació usant el mètode de *Test-Driven Development*. Eines com *HUnit* permetran escriure una prova que verifiqui el format d'avaluació correcta segons les normes de GHC abans d'implementar el detall de l'algorisme monàdic corresponent.
- **Cerimònies de Validació Externa (Sprint Reviews):** Cada quinzenal es realitzarà una reunió presencial o telemàtica amb el Director de Projecte de la FIB. En aquestes tutories s'ensenyarà exclusivament programari funcionant, es discutirà el *feedback* pedagògic de la interfície generada i es consensuarà l'enfocament i els possibles plans de mitigació del Sprint vinent. Aquest *loop* de realimentació és crucial per alinear el projecte amb el pla educatiu establert de forma pragmàtica.

0.5 Planificació Temporal

0.5.1 Gestió del Calendari i Distribució d'Hores

El TFG de la Facultat d'Informàtica de Barcelona consta de 18 crèdits ECTS. Assumint una dedicació de 30 hores per crèdit, el projecte té una **durada total de 540 hores de dedicació de l'estudiant**. El projecte s'executa oficialment des del **15 de febrer de 2026 fins al 15 de juny de 2026** (17 setmanes totals d'execució, on l'última setmana es destina exclusivament a la defensa). Per assumir les 540 hores, la distribució de dedicació de l'estudiant serà d'aproximadament 32 hores setmanals de mitjana, que s'ajustaran depenent de la intensitat de la fase (major dedicació durant la implementació del nucli tecnològic).

0.5.2 Descripció Detallada de les Tasques, Precedències i Rols

S'ha abandonat l'agrupació per fases genèriques per detallar un a un els elements atòmics de treball. S'assignen diferents "rols" a l'estudiant per facilitar posteriorment la imputació econòmica.

Taula 2: Taula detallada de tasques, durada, precedències i rols associats

ID	Nom de la Tasca	Hores	Preced.	Rol Assignat	Recurs Material
T1	Gestió del Projecte (GEP)	110h			
T1.1	Lliuraments i Context GEP	30	-	Gestor Proj.	Biblio/Office
T1.2	Planificació i Gantt	20	T1.1	Gestor Proj.	LaTeX
T1.3	Pressupost i Sostenibilitat	20	T1.2	Gestor Proj.	Excel
T1.4	Redacció Memòria TFG	40	T1.1	Gestor Proj.	LaTeX
T2	Recerca i Base Teòrica	65h			
T2.1	Estudi Semàntica i GHC	35	-	Eng. Programari	Documents
T2.2	Investigació Parsers Monàdics	30	-	Eng. Programari	PC
T3	Disseny Arquitectònic	65h			
T3.1	Disseny Gramàtica i AST	35	T2.1	Eng. Programari	PC
T3.2	Mockups GUI	30	-	Eng. Programari	Figma

ID	Nom de la Tasca	Hores	Preced.	Rol Assignat	Recurs Material
T4	Implementació Core	160h			
T4.1	Desenvolupament Lexer	30	T3.1	Eng. Programari	GHC, Git
T4.2	Desenvolupament Parser	40	T4.1	Eng. Programari	GHC, Git
T4.3	Motor de Reducció Mandro- sa	60	T4.2	Eng. Programari	GHC, Git
T4.4	Captura i Historial d'Estats	30	T4.3	Eng. Programari	GHC, Git
T5	Frontend i Qualitat	140h			
T5.1	Implementació UI	50	T3.2	Eng. Programari	IDE Web
T5.2	Connexió UI - Haskell Mo- tor	40	T4.4,T5.1	Eng. Programari	IDE Web
T5.3	Bateria Proves Unitàries	30	T4.3	Analista QA	HUnit
T5.4	Proves Integració i Docs	20	T5.2	Analista QA	Git
T6	Defensa Pública	Total	540h		
T6.1	Preparació material Defen- sa	0h	T5.4	Gestor Proj.	Ppoint

0.5.3 Recursos Humans i Materials

La realització de les tasques prèvies implica:

- **Recursos Humans (RRHH):** Es defineixen 3 rols principals que assumeix l'estudiant: *Gestor de Projectes* (responsable de GEP i Memòria), *Enginyer de Programari* (recerca, disseny i programació) i *Analista de QA* (proves). A més, el *Director del TFG* actua com a validador de qualitat invertint unes 15 hores globals en reunions de seguiment.
- **Recursos Materials:** Ordinador portàtil personal, compilador de Haskell (GHC), gestor de paquets Stack, plataformes de repositoris (GitHub), programari ofimàtic i de disseny gràfic per a mockups (Figma) i llibreries de codi obert gratuïtes.

0.5.4 Diagrama de Gantt

S'ha modelat el diagrama de Gantt respectant les dependències marcades a la taula anterior, assegurant que les activitats de recerca i documentació es solapen de manera

concurrent i realista amb les de desenvolupament, atès que l'estudiant distribueix la seva jornada.

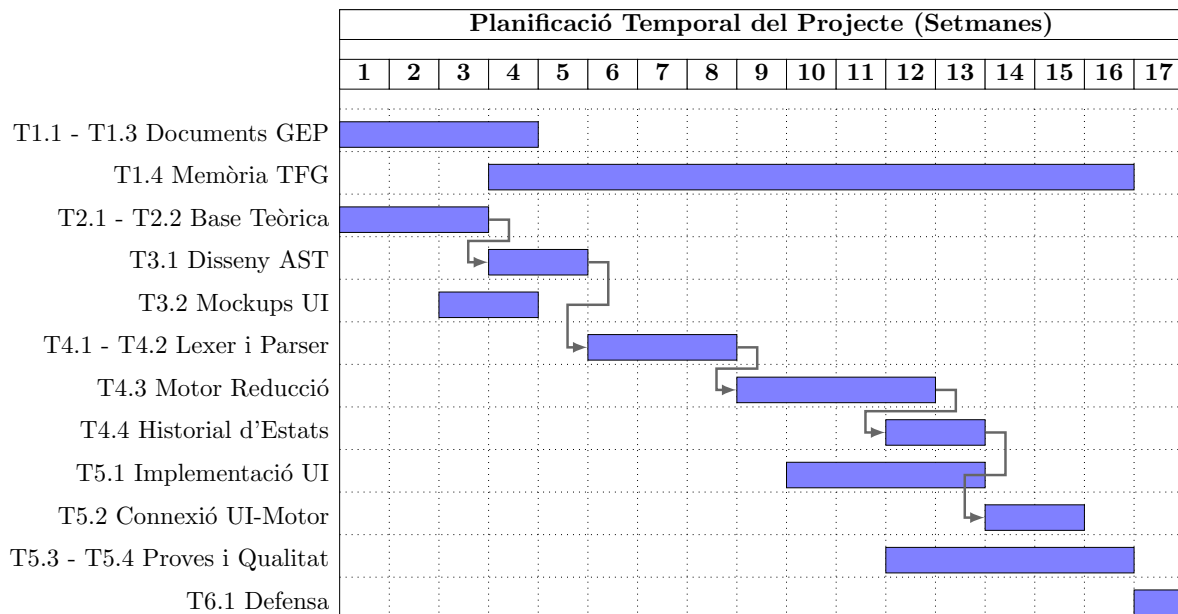


Figura 1: Diagrama de Gantt detallat amb concurrències i la ruta crítica d'implementació.

0.6 Pressupost i Sostenibilitat

Aquest apartat quantifica la viabilitat econòmica del projecte assignant valors als rols definits en la planificació temporal. També s'estableix un control de desviacions rigorós i s'avaluen les tres dimensions de sostenibilitat exigides.

0.6.1 Identificació i Justificació de Costos

Per dissenyar el pressupost d'una manera fidel a un entorn empresarial s'imputen quatre tipus de costos: Costos directes de personal, Amortitzacions de material, Despeses generals i Fons de seguretat (contingències i imprevistos).

Costos Directes de Personal per Tasca

Tot i ser un projecte acadèmic de l'alumne, se simulen els costos laborals. Els salaris per hora s'han justificat utilitzant dades de l'Acord Marc del sector TIC d'Espanya i portals com Glassdoor (dades referenciades per a l'any en curs):

- **Gestor de Projectes (Project Manager):** 35 €/hora. Gestiona documentació i fites.
- **Enginyer de Programari (Software Engineer):** 25 €/hora. Disseny i picar codi.
- **Analista de Qualitat (QA):** 20 €/hora. Creació de tests i validació d'errors.

A continuació es detalla el cost específic per cada activitat basat en la Taula 2:

Taula 3: Costos de Personal per Tasques del Gantt

Tasca	Rol Responsable	Hores	Cost Total (€)
T1. Gestió del Projecte (GEP)	Gestor de Projectes	110h	3.850,00 €
T2. Recerca i Base Teòrica	Eng. Programari	65h	1.625,00 €
T3. Disseny Arquitectònic	Eng. Programari	65h	1.625,00 €
T4. Implementació Core	Eng. Programari	160h	4.000,00 €
T5.1 i T5.2 (Front i Connexió)	Eng. Programari	90h	2.250,00 €
T5.3 i T5.4 (Proves QA)	Analista de Qualitat	50h	1.000,00 €
Total Personal directe	-	540h	14.350,00 €

Costos Generals i Amortitzacions

Per al hardware, s'estima una vida útil de l'ordinador portàtil de treball de 4 anys (48 mesos). Sent el seu valor de 2.000€ i utilitzant-se durant els 4 mesos del projecte, l'amortització és de **166,66 €**. Tot el programari utilitzat és lliure i de cost zero. Les despeses indirectes d'oficina (Internet i consum elèctric de l'equip) s'estimen en una tarifa plana imputada de **100,00 €** per tot el cicle.

Estimació del Pressupost Total

S'aplica un 15% de Fons de Contingència per cobrir riscos documentats al Gantt i un 5% d'imprevistos fora del control de l'abast.

Taula 4: Pressupost Global i Totalitzat

Concepte (Partida)	Cost Estimad
Total Costos de Personal	14.350,00 €
Amortització Maquinari	166,66 €
Despeses Generals (Indirectes)	100,00 €
Suma Base del Projecte	14.616,66 €
Contingències (15%)	2.192,50 €
Imprevistos (5%)	730,83 €
PRESSUPOST FINAL TOTAL	17.539,99 €

0.6.2 Control de Gestió de Desviacions

Amb l'objectiu d'assegurar l'excel·lència operativa i evitar desviacions econòmiques respecte a l'estimació anterior, s'implementarà la tècnica **EVM (Earned Value Management)**. Aquesta tècnica es monitoritzarà quinzenalment. S'utilitzaran tres mètriques base:

- **Valor Planificat (PV):** Cost estimat de les tasques que, segons el Gantt, haurien d'estar finalitzades.
- **Cost Real (AC):** Hores invertides realment per l'estudiant multiplicades pel salari del seu rol actiu.
- **Valor Guanyat (EV):** Cost original pressupostat de la part de les tasques que realment s'ha completat fins avui.

Els indicadors numèrics facilitadors del control seran:

- **SV (Variació del Calendari) = EV - PV.** Un valor inferior a 0 indica retard sobre el Gantt.
- **CV (Variació del Cost) = EV - AC.** Un valor inferior a 0 indica sobrecost econòmic.

Si l'índex CPI (EV/AC) cau per sota de l'1.0 durant les fases centrals, es destinarà partida del Fons de Contingència i, de ser necessari, es cancel·laran requeriments no funcionals secundaris per retornar al pressupost d'equilibri.

0.6.3 Informe de Sostenibilitat

Abans de la redacció d'aquesta memòria s'ha completat la matriu i l'enquesta d'autoavaluació de Sostenibilitat EDINSOST requerida per la facultat. Les conclusions per a les 3 dimensions són:

- **Dimensió Econòmica:** És un projecte altament viable. El seu desenvolupament i explotació es realitza amb programari completament de codi obert i no requereix servidors d'alt rendiment per ser allotjat, cosa que eximeix la universitat o l'usuari final de pagar cap tipus de llicència o subscripció futura.
- **Dimensió Ambiental:** La petjada ecològica és mínima i està circumscrita únicament als 32 kWh (aproximadament) consumits per l'ordinador portàtil de l'estudiant durant les 540 hores d'execució.
- **Dimensió Social:** El component d'impacte social és enorme. Millora radicalment l'entorn docent, afavoreix la inclusió acadèmica disminuint l'elevada corba d'aprenentatge del paradigma funcional i prevé l'abandonament o la frustració d'estudiants. La futura publicació del codi de forma lliure fomenta el coneixement compartit globalment.

Part II

Desenvolupament Tècnic

Capítol 1

Anàlisi Lèxica i Sintàctica

L'anàlisi d'un programa font es divideix tradicionalment en dues fases clarament diferenciades: l'anàlisi lèxica (*lexing*) i l'anàlisi sintàctica (*parsing*). En el disseny d'aquest intèrpret, s'ha optat per defugir l'ús d'eines externes generadores de codi basades en taules estàtiques com *Alex* o *Happy*. En el seu lloc, s'ha implementat un enfocament natiu basat en **combinadors de parsers** encastrats directament en el llenguatge amfitrió, Haskell.

Aquesta decisió de disseny, fortament fonamentada en els treballs sobre combinadors monàdics de Hutton i Meijer [4], permet mantenir l'homogeneïtat absoluta de la base de codi, aprofita el potent sistema de tipus de Haskell per garantir la seguretat en temps de compilació i simplifica dràsticament l'arquitectura de l'intèrpret al no requerir passos de compilació intermedis per a la sintaxi.

Per tal d'il·lustrar de manera didàctica i cohesiva el funcionament intern del sistema, aquesta memòria utilitzarà una **expressió guia** que ens acompanyarà de manera transversal al llarg de tots els capítols. Analitzarem el “viatge” complet d'aquesta línia, des que és una simple cadena de text pla fins a la seva reducció final en la màquina de grafs. L'expressió triada és:

$$(\backslash x\ y \rightarrow x * y)\ 2\ 3$$

En aquest capítol, s'exposa com aquesta seqüència de caràcters es transforma en una estructura de dades arbòria que reflecteix la semàntica formal del llenguatge.

1.1 Disseny de la Gramàtica i l'Arbre de Sintaxi Abstracta

L'objectiu primer d'aquesta fase és construir una representació abstracta en forma d'arbre (AST, *Abstract Syntax Tree*) de qualsevol codi subministrat per l'usuari. El tipus de dades algebraic central que governa el nucli del llenguatge és `Expr`, definit al mòdul `HSTipus`:

```
1 data Expr = Val Int
2           | Var String
3           | App Expr Expr
4           | Lam String Expr
5           | If Expr Expr Expr
6           deriving (Show, Eq)
```

Listing 1.1: Definició del tipus algebraic d'expressions (`Expr`)

1.1.1 Suficiència Teòrica del Nucli d'Expressions

Aquest conjunt de cinc nodes és sorprenentment minimalista però completament suficient per formalitzar un llenguatge de programació funcional complet. El fonament teòric d'aquesta suficiència rau en el **Càlcul Lambda** pur, el qual formalment només requereix tres constructors fonamentals: variables (`Var`), abstraccions lambda (`Lam`) i aplicacions de funcions (`App`). Com és ben conegut, aquest nucli és Turing complet i pot modelar qualsevol estructura de control o de dades mitjançant codificacions estructurals.

No obstant això, tal com argumenta Simon Peyton Jones en el seu text clàssic sobre la implementació de llenguatges funcionals [5], un intèrpret pràctic requereix un **Càlcul Lambda Enriquit** (*Enriched Lambda Calculus*) per evitar la ineficiència extrema de les codificacions pures de Church. Per aquest motiu, s'ha estès el nucli algebraic de l'AST amb dues addicions semàntiques primitives integrades de forma nativa:

- **Val Int:** Proporciona una representació directa dels enters de la màquina i en permet el càlcul numèric immediat. Per mantenir l'AST simple, els operadors aritmètics i lògics (com `*` o `+`) no disposen d'un node propi, sinó que es tracten formalment com a variables (`Var`) el tipus de les quals està mapejat globalment a l'entorn de primitives o *Prelude*, d'acord amb el disseny suggerit a [5].
- **If Expr Expr Expr:** Introdueix una estructura de control condicional que s'executa aprofitant la mandra (*laziness*) inherent del motor d'avaluació. El node avalua

estrictament la condició i, posteriorment, només redueix la branca escollida, evitant camins d'execució infinits o divergents.

1.1.2 El Tractament de la Recursivitat mitjançant Supercombinadors

Un detall arquitectònic crucial heretat dels esquemes clàssics de compilació funcional [5, 7] és l'absència d'un node de recursivitat dedicat dins de **Expr** (com podria ser un constructor **LetRec** o un operador explícit de punt fix **Fix**). En aquest projecte, la recursivitat es gestiona de manera diferida i global mitjançant la transformació del codi en **Supercombinadors**.

Quan el parser detecta definicions de funcions auto-referencials o mútuament recursives, les lliga directament a l'entorn de declaracions d'alt nivell. Més endavant, durant la generació de codi i instanciació, el motor tradueix aquestes declaracions en supercombinadors globals de manera que els cicles i la recursivitat es resolien directament mitjançant l'enllaç dinàmic de punters al *heap* [7]. Aquest enfocament no només manté l'AST net, sinó que alleugereix substancialment la implementació de l'algorisme d'inferència de tipus basat en el sistema Damas-Milner [1].

1.2 L'Abstracció de Declaracions (*Bindings*)

Un programa funcional real no es limita a una única expressió aïllada; s'organitza com una seqüència estructurada de definicions globals seguides de l'expressió que l'usuari desitja computar. Per modelar adequadament aquest flux, el parser encapsula les definicions en el tipus **Binding**, governat pels combinadors estructurals **program** i **line**:

```
1 data Binding = Bind String Expr deriving (Show)
2 type Program = ([Binding], Expr)
```

Listing 1.2: Estructures d'enllaç i flux del programa

El combinador **binding** actua com l'eix de **desugarejament** (*desugaring*), prenent sintaxi altament abstracta i refinant-la cap al nucli minimalista de l'AST [5]. Aquesta abstracció suporta de manera directa dues tècniques expressives clau de la programació funcional:

1. **Definicions Multiargument i Currificació:** Permet processar línies de codi nates de l'estil `suma x y = x + y`. L'analitzador recull de forma seqüencial la

llista d'arguments post-identificador i recorre a la funció interna `lambdifica`, la qual nia de manera iterativa constructors `Lam`. L'enllaç resultant es transforma conceptualment en `Bind "suma"(Lam "x"(Lam "y"(...)))`.

2. **Guàrdies Sintàctiques (*Guards*):** Reconeix estructures condicionals alternatives mitjançant el caràcter d'afix `|`. El combinador absorbeix un patró de condicions usant `guardElement` juntament amb la clàusula de tancament obligatòria `finalGuardElement` (associada a la paraula clau `otherwise`). Mitjançant la funció de reducció sintàctica `ififica`, aquesta seqüència es recodifica com un niu altament ordenat d'expressions primitives `If`, imitant els mètodes de traducció semàntica formalitzats per Peyton Jones [5].

Per tal de garantir la robustesa del sistema i evitar que l'usuari redefineixi identificadors interns que trenquin la coherència del parser, la introducció de variables està filtrada de forma estricta per les llistes `paraulesProhibidesBind` i `paraulesProhibides`. Qualsevol intent d'utilitzar tokens estructurals (com `if`, `then`, `else`, `True` o `otherwise`) com a identificadors d'arguments o noms de funció és immediatament avortat pel combinador a través de la fallida monàdica `empty`.

1.3 Implementació i Arquitectura del Parser Monàdic

1.3.1 Fonamentació Teòrica i la Mònada Parser

Seguint els corrents teòrics consolidats per Philip Wadler sobre la utilització d'estructures monàdiques per a l'aïllament d'efectes col·laterals [8], un parser es defineix com una computació que rep una cadena d'entrada i retorna una llista de parells possibles. Cada parell representa un èxit en el parsatge, desant el valor transformat juntament amb el subsegment de text restant:

```
1 newtype Parser a = Parser {parse :: String -> [(a, String)]}
```

Listing 1.3: Definició monàdica del tipus Parser

Aquest disseny modela el no-determinisme mitjançant el patró conegut com a “llista de l'èxit” (*list of successes*) [8]. La instanciació de la classe `Monad` defineix l'operador d'enllaç seqüencial `>>=` mitjançant llistes per comprensió, encarregant-se de propagar automàticament l'estat del flux de caràcters d'una subfunció a la següent:

```
1 instance Monad Parser where
2   return = pure
```

```

3 p >>= f = Parser $ \inp ->
4   concat [parse (f v) inp' | (v, inp') <- parse p inp]

```

Listing 1.4: Instanciació de la infraestructura de classes de tipus per al Parser

1.3.2 Els Maons Sintàctics: Parsers Monàdics Primitius

Per comprendre el funcionament intern de l'analitzador, cal descendir cap als combinadors elementals implementats al mòdul. Tot el sistema d'anàlisi de patrons sintàctics complexos es construeix per composició a partir de tres parsers atòmics de caràcters: `result`, `zero` i `item`.

El combinador `item` és l'única funció que té accés directe i destructiu a la cadena de text d'entrada. Consumeix de forma incondicional el primer caràcter disponible, actuant com el motor d'avanç elemental de l'analitzador:

```

1 item :: Parser Char
2 item = Parser parseItem
3   where
4     parseItem [] = []
5     parseItem (x : xs) = [(x, xs)]

```

Listing 1.5: Implementació de la primitiva elemental `item`

La veritable potència d'aquest enfocament declaratiu es fa palesa en introduir el combinador `sat` (*satisfy*), el qual actua com un filtre sintàctic. Aquest mètode rep un predicat booleà i utilitza l'operador d'enllaç monàdic (`>>=`) per extreure el caràcter prèviament consumit per `item`:

```

1 sat :: (Char -> Bool) -> Parser Char
2 sat p = item >>= \x -> if p x then result x else zero

```

Listing 1.6: Combinador monàdic condicional `sat`

Si el caràcter capturat `x` satisfà la condició del predicat `p x`, s'encapsula de nou mitjançant `result x` (mantenint l'estat actual de la cadena); en cas contrari, es desvia cap a `zero`. El parser `zero` injecta la llista buida `[]`, provocant de forma immediata la fallida d'aquesta línia de còmput i forçant el mecanisme de *backtracking*. A partir d'aquesta elegant combinació monàdica abstracta, es deriven funcionalment tots els reconeixadors de caràcters d'alt nivell del sistema sense haver de desmuntar mai més la tupla de text de manera manual [4]:

```

1 digit :: Parser Char
2 digit = sat isDigit

```

```

3
4 lletra :: Parser Char
5 lletra = sat isLetter

```

Listing 1.7: Derivació declarativa de parsers de caràcters

1.3.3 Gestió de l'Entorn i Aïllament de l'Espai en Blanc: El combinador token

En els dissenys de compiladors tradicionals, els espais en blanc i els comentaris es depuren en una fase de *lexing* independent per evitar embrutar les regles sintàctiques. En utilitzar combinadors de parsers, aquestes dues fases s'integren de manera harmoniosa en un únic pas gràcies a l'abstracció d'estat proporcionada per les mònades i els aplicatius.

Per aconseguir aquest aïllament d'estil en el nostre intèrpret, s'han definit els mètodes `comentari`, `blancOComentari` i el combinador d'indirecció `token`. El parser `token` actua com un patró d'envolcall (*wrapper*) que s'assegura de netejar completament qualsevol soroll visual (com tabuladors, salts de línia o comentaris iniciats per `--`) abans i després de processar el nucli semàntic central d'un element. La seva definició aprofita els operadors de seqüenciació de la classe `Applicative`:

```

1 espais :: Parser [()]
2 espais = mul blancOComentari
3
4 token :: Parser a -> Parser a
5 token p = espais *> p <*> espais

```

Listing 1.8: Neteja de traces i comentaris mitjançant el combinador token

L'ús de l'operador `*>`, heretat de la instanciació estructural de `Alternative` i `Applicative`, indica al parser que ha d'executar primer `espais` (consumint i descartant la brossa de text inicial), després executar el parser passat com a paràmetre `p` (conservant-ne el resultat de retorn), i finalment tornar a cridar `espais` per via de `<*>` (netejant el text posterior). Gràcies a `token`, mètodes estructurals com `intToken`, `compParser` o els mateixos literals de variables poden ignorar completament si l'usuari ha inserit espais extra o salts de línia en l'expressió original.

1.3.4 Gestió de Precedència i Descens Sintàctic Jerarquitzat

L'ambigüitat és un dels problemes endèmics en l'anàlisi de gramàtiques complexes que contenen operadors infixes combinats amb l'aplicació per juxtaposició de termes. Per re-

ordre de manera determinista la **precedència d'operadors**, s'ha dissenyat un esquema de descens jeràrquic molt rígid [5], on cada nivell de prioritat està aïllat en una funció pròpia que enllaça amb el nivell immediatament superior.

El punt de partida de l'anàlisi d'expressions és el combinador `expr`. Aquest delega de manera controlada el flux de dades de menor a major precedència sintàctica:

```
expr → lam / ifThenElse → operator → logicOr → logicAnd → comp → suma →
      term → ap → atom
```

Prenent com a referència la nostra expressió guia, quan l'analitzador processa el cos interior del bloc funcional `x * y`, el flux de crides descendeix a través de tota la jerarquia fins a topar-se amb el combinador `term`, el qual s'encarrega de detectar l'operador multiplicatiu `*` mitjançant `mulDivParser`. Això força que les multiplicacions agrupin els seus operands de forma prioritària abans que operacions com la suma (`suma`) o les comparacions lògiques puguin segrestar els nodes de l'AST, garantint la preservació exacta de la intencionalitat semàntica del programador.

1.3.5 Aplicació de Funcions i Currificació Sintàctica

D'acord amb els canons de disseny dels llenguatges de la família del Càlcul Lambda recollits per Peyton Jones [5], l'aplicació de funcions és estructuralment **associativa per l'esquerra**. L'expressió guia `(\x y -> x * y) 2 3` no denota un bloc de codi que rep un vector homogeni de dos paràmetres, sinó que es tradueix formalment en la computació seqüencial `((\x y -> x * y) 2) 3`.

El combinador `ap` resol aquesta agrupació recollint en primera instància un `atom` base (en el nostre cas, la definició abstracta de la funció lambda) i absorbint descol·legiadament una llista de possibles àtoms adjacents a la seva dreta mitjançant `mul atom` (el qual retornarà la llista `[Val 2, Val 3]`). Un cop recollit el bloc complet, la funció interna `insert` executa un plec estructural cap a l'esquerra (*left-fold*), niant asimètricament constructors `App`:

```
1 ap :: Parser Expr
2 ap = do
3     x <- atom
4     ys <- mul atom
5     return (insert (x:ys))
6         where
7             insert :: [Expr] -> Expr
```



```

8      insert [e] = e
9      insert exprs = (App (insert (init exprs)) (last exprs))

```

Listing 1.9: Associativitat per l'esquerra en el combinador d'aplicació

Aquest procés d'implicació geomètrica es comporta a la inversa quan analitzem la definició de paràmetres d'una funció. El combinador **lam** processa la declaració de variables associant la seva estructura cap a la **dreta**. Durant l'anàlisi del component `\x y -> x * y`, el mètode absorbeix els identificadors de variables presents en la capçalera de la funció mitjançant `mes nomVariable` i reconstrueix la cua d'execució de l'AST de manera imbricada, generant el niu `Lam "x"(Lam "y"(...))`.

1.4 Arbres de Sintaxi Abstracta Resultant

Un cop el combinador d'alt nivell **program** ha consumit amb èxit la totalitat dels caràcters de l'entrada i ha validat que no queden fragments residuals sense analitzar, el sistema retorna l'Arbre de Sintaxi Abstracta definit.

Com es pot visualitzar de manera intuïtiva a la Figura 1.1, l'impacte geomètric derivat de les regles de currificació defensades per la teoria funcional [5] es plasma perfectament en l'arquitectura de l'arbre generat: els nodes d'aplicació **App** s'inclinen de manera orgànica cap a l'esquerra de la jerarquia. El valor de tancament `Val 3` es manté en el node arrel superior, lligat directament a la computació intermèdia que prèviament havia unit l'expressió lambda original amb l'operand `Val 2`. Amb la construcció d'aquest graf abstracte i net, conclou oficialment la fase sintàctica, facilitant l'estructura requerida per a l'execució del motor d'inferència de tipus.

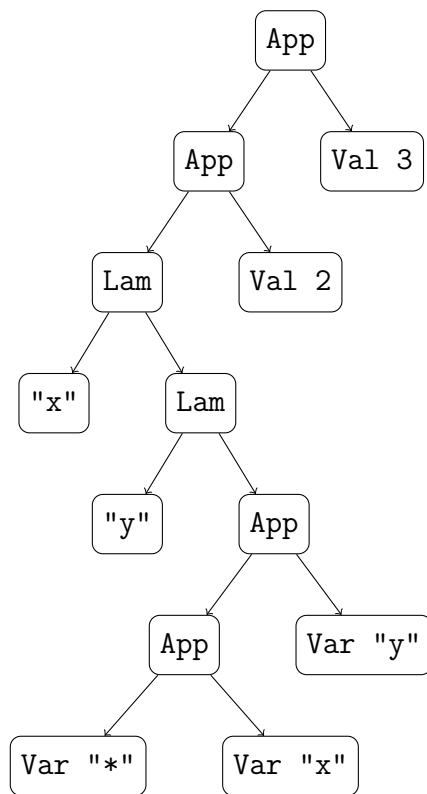


Figura 1.1: Arbre de Sintaxi Abstracta (AST) generat pel parser per a l'expressió guia `(\x y -> x * y) 2 3`.

Capítol 2

Sistema de Tipus

2.1 Fonaments de l’Algorisme de Hindley-Milner

El sistema de tipus d’aquest intèrpret s’ha dissenyat sota el marc formal de l’algorisme de **Hindley-Milner (HM)**, un sistema de deducció lògica adaptat per a llenguatges funcionals amb polimorfisme paramètric. Com descriu l’obra de referència de Peyton Jones [5], la potència d’aquest enfocament rau en el fet que el programador gaudeix de la robustesa d’un llenguatge de tipat estàtic fort sense patir la càrrega sintàctica de realitzar anotacions de tipus explícites a cada declaració.

L’algorisme opera sota un principi d’obvencions axiomàtiques i deduccions estructurals. El sistema calcula formalment el *principal type* (el tipus més general possible), basant-se en les teories formulades originalment per Luis Damas i Robin Milner [1]. Aquest rigor matemàtic garanteix el teorema de seguretat fonamental formulat pel mateix Milner: “*Well-typed programs cannot go wrong*”. En el context d’aquest projecte, que un programa estigui “ben tipat” significa que el motor d’inferència codificat a `HSInferenciaTipus.hs` és capaç de trobar una assignació de tipus consistent que respecti les regles sintàctiques primitives. Si l’algorisme detecta una incongruència semàntica, el procés d’execució s’interromp immediatament abans de la fase de reducció de grafs, protegint la integritat del sistema de memòria.

La sintaxi dels tipus suportats es modela de forma abstracta en el fitxer `HSTipus.hs` mitjançant el tipus de dades algebraic `Tipus`:

```
1 data Tipus = TInt
2           | TBool
3           | TFun Tipus Tipus
4           | TVar String
```

5 `deriving (Show, Eq)`

Listing 2.1: Definició formal dels tipus de dades de l'interpret

Aquest model reflecteix fidelment els elements atòmics del llenguatge: els tipus base concrets (TInt per a literals numèrics i TBool per a valors booleans), el constructor de funcions d'ordre superior (TFun), i les variables de tipus abstractes (TVar), que actuen com a marcadors de posició universals durant les fases d'unificació.

2.2 L'Algorisme d'Unificació de Robinson i el Test d'Ocurrencia

L'engranatge central que permet resoldre les equacions de tipus generades per l'interpret és l'algorisme d'unificació de Robinson. Donats dos tipus qualssevol τ_1 i τ_2 , el propòsit d'aquest algorisme és trobar un conjunt de mapatges —anomenat Substitució (σ)— que aconseguixi que ambdós tipus esdevinguin estructuralment idèntics quan se'ls aplica la dita transformació ($\sigma(\tau_1) = \sigma(\tau_2)$).

A la nostra implementació, aquest comportament es troba encapsulat en la funció `generaSubst`. Aquesta funció opera comparant la geometria d'ambdues estructures de dades, resolent dependències creuades i evitant cicles infinits mitjançant un rigorós *Occurs Check* (Test d'Ocurrencia):

```
1 generaSubst :: Tipus -> Tipus -> Either String Subst
2 generaSubst TInt TBool = Left "Error d'inferència: No es pot unificar
   un enter amb un booleà"
3 generaSubst TBool TInt = Left "Error d'inferència: No es pot unificar
   un booleà amb un enter"
4
5 generaSubst (TFun t1 t2) (TFun t3 t4) = do
6   first  <- generaSubst t1 t3
7   second <- generaSubst (aplicaSubst first t2) (aplicaSubst first t4)
8   return (second ++ first) -- Posem les més recents primer
9
10 generaSubst (TVar ('t':s1)) (TVar s2) = Right [((('t':s1), (TVar s2)))]
11 generaSubst (TVar s1) (TVar s2) = Right [(s2, (TVar s1))]
12
13 generaSubst t (TVar s)
14   | TVar s == t = return []
15   | ocorreEn s t = Left "Error d'inferència: Tipus infinit"
16   | otherwise = return [(s,t)]
17
```

```

18 generaSubst (TVar s) t = generaSubst t (TVar s)
19 generaSubst _ _ = Right []
20
21 ocorreEn :: String -> Tipus -> Bool
22 ocorreEn s (TVar a) = s == a
23 ocorreEn s (TFun t1 t2) = ocorreEn s t1 || ocorreEn s t2
24 ocorreEn _ _ = False

```

Listing 2.2: Implementació del motor d'unificació de Robinson amb protecció contra tipus infinits

L'algorisme es divideix en els següents escenaris operacionals clars:

1. **Conflicte de Tipus Primitius:** L'intent directe d'equiparar `TInt` amb `TBool` (o viceversa) representa una violació semàntica irreconciliable. La funció retorna una fallida monàdica `Left`, propagant un missatge clar d'error cap a la terminal de l'usuari.
2. **Descomposició d'Estructures (TFun):** Quan es comparen dues funcions, l'algorisme es ramifica recursivament. És de vital importància notar com primer s'unifiquen els dominis (`t1` i `t3`) generant la substitució `first`. A continuació, per mantenir la coherència lògica, **s'aplica immediatament** aquesta substitució `first` als codominis (`t2` i `t4`) abans d'intentar unificar-los. Això garanteix que qualsevol resolució de variables feta en els arguments es propagui cap als resultats.
3. **El Test d'Ocurrencia (*Occurs Check*):** Un dels reptes teòrics més complexos de la inferència és la gestió d'expressions divergents com $\lambda x. x \rightarrow x$. Intentar inferir això implicaria dir que el tipus de `x` ha de ser igual a `x -> y`. Això genera un tipus infinit: $(\dots \rightarrow y) \rightarrow y$. Per evitar que l'unificador entri en un bucle infinit penjant l'interpret, s'ha dissenyat la funció auxiliar `ocorreEn`. Aquesta funció fa un escaneig profund de l'arbre: si s'intenta lligar una variable `s` a un tipus `t` que *ja conté* la pròpia variable `s` en el seu interior, el sistema llança l'error "Tipus infinit", avortant la operació de manera segura.

2.3 El Motor d'Inferència Estructural

La deducció de tipus s'executa mitjançant un recorregut sintàctic recursiu sobre l'AST utilitzant la funció `infereix`. En aquesta implementació evolucionada, la funció requereix la gestió simultània de dos contextos: el context inicial global (`inictx`), on viuen les primitives del *Prelude*, i el context lèxic local (`ctx`), que muta a mesura que s'aprofundeix

en les expressions. Els contextos es modelen com a llistes de parells: `type Context = [(String, Tipus)]`.

La propagació d'informació es realitza mitjançant la composició i l'aplicació seqüencial de substitucions. Les funcions auxiliars `aplicaSubst` i `aplicaSubstCtx` s'encarreguen de recórrer un tipus o un context sencer per substituir-ne les variables lliures per les noves definicions estructurals descobertes en temps de computació:

```

1 aplicaSubst :: Subst -> Tipus -> Tipus
2 aplicaSubst s (TVar v) = case lookup v s of
3     Just t  -> aplicaSubst s t -- Continuem
4     Nothing -> TVar v
5 aplicaSubst s (TFun t1 t2) = TFun (aplicaSubst s t1) (aplicaSubst s t2)
6 aplicaSubst _ t = t
7
8 aplicaSubstCtx :: Subst -> Context -> Context
9 aplicaSubstCtx _ [] = []
10 aplicaSubstCtx subs ((st,t):ctx) = ((st, aplicaSubst subs t): (
    aplicaSubstCtx subs ctx))

```

Listing 2.3: Mecanisme de propagació de substitucions en tipus i contextos

La funció `aplicaSubst` realitza una cerca recursiva profunda (*deep lookup*): si una variable `v` es mapeja cap a una altra variable de tipus, l'algorisme continua explorant la llista de substitucions iterativament per assegurar-se que es resol el tipus terminal més madur disponible a la memòria.

2.3.1 Polimorfisme Paramètric i Instanciació de Variables Fresques

Un dels assoliments més destacats del disseny d'aquest motor és la implementació de polimorfisme paramètric (*Lambda-Expression Polymorphism*) per a funcions predefinides [5]. Quan es fa una crida a una funció polimòrfica emmagatzemada al `preludeTypeEnv` (com ara `id :: a -> a`), no podem utilitzar la variable "a" directament cada vegada. Si ho féssim, intentar avaluar `if (id True) then (id 42) else 0` faria que l'algorisme unifiqués "a" primer amb `Bool` i després, de manera contradictòria, amb `Int`, llançant un error fatal.

Per solucionar això, cada vegada que s'extreu un tipus del context, s'ha de **instanciar**: cal substituir les seves variables genèriques originals per "variables fresques" noves, úniques per a aquella crida en concret. Això s'ha solucionat mitjançant una enginyosa estratègia d'acumulació de números reflectida en les funcions `renameTVar` i `forceRenameTVar`:

```

1 renameTVar :: Tipus -> Int -> Maybe Tipus
2 renameTVar (TFun t1 t2) n = do
3   r1 <- (renameTVar t1 n)
4   r2 <- (renameTVar t2 n)
5   return (TFun r1 r2)
6 renameTVar (TVar ('t':rest)) n = Just (TVar ('t':rest))
7 renameTVar (TVar s) n = Just (TVar (s++(show n)))
8 renameTVar t _ = Nothing
9
10 forceRenameTVar :: Tipus -> Int -> Maybe Tipus
11 forceRenameTVar (TFun t1 t2) n = do
12   r1 <- (forceRenameTVar t1 n)
13   r2 <- (forceRenameTVar t2 n)
14   return (TFun r1 r2)
15 forceRenameTVar (TVar s) n = Just (TVar (s++(show n)))
16 forceRenameTVar t _ = Nothing

```

Listing 2.4: Mètodes per a l'instanciació de variables i reanomenament dinàmic

El mecanisme fa servir un comptador enter global (n) que viatja a través de la mònada. Quan un tipus genèric és sol·licitat, aquestes funcions recorren el tipus de dades algebraic. Si troben una variable com `TVar "a"`, hi concatenen el comptador vigent, transformant-la en `TVar "a1"`. Així, crides successives generaran signatures aïllades com `a1 -> a1` o `a2 -> a2`, prevenint qualsevol col·lisió o pol·lució de l'espai de noms de tipus de la memòria de l'interpret. Cal destacar que `renameTVar` fa una excepció lògica per a les variables que comencen per `'t'`, ja que aquestes són reconegudes com a variables internes de treball de l'algorisme en curs i no s'han de duplicar, en contrast amb `forceRenameTVar` que força l'agrupació sense distincions per a lligams preexistents.

2.3.2 Anàlisi de Casos Claus de la funció infereix

L'arquitectura funcional de la deducció estructural s'implementa bifurcant el patró sintàctic de l'expressió analitzada. Els blocs monàdics `do` gestionen de manera nativa les fallides d'unificació i l'enllaç de la mònada interna de Haskell [8].

Cerca de Variables (Variables Lliures i Lligades)

Quan l'algorisme analitza un node `Var a`, fa una exploració de presència. Primerament busca la variable en l'entorn local (`ctx`); si hi és, significa que és una variable capturada localment (per exemple el paràmetre d'una lambda externa). Si no es troba, procedeix a cercar-la a l'entorn global (`inictx`). És precisament aquí on fa ús de les funcions

descrites anteriorment per a instanciar el seu polimorfisme de forma segura, modificant adequadament el comptador d'adreces n .

Abstraccions Lambda i Paràmetres Pattern-Matched

El tractament de les expressions `Lam s e` presenta una dualitat fonamental segons la naturalesa del paràmetre d'entrada:

```

1 infereix inctx ctx (Lam s e) n = do
2   case all isDigit s of
3     True  -> (do
4       t_s <- Right (TInt)
5       (t1, s1, n1) <- infereix inctx ctx e n
6       t_retorn <- Right (aplicaSubst s1 (TFun t_s t1))
7       return (t_retorn, s1, n1)
8     )
9     False -> (do
10      t_s <- Right (TVar ("t"++ show n))
11      (t1, s1, n1) <- infereix inctx ((s,t_s):ctx) e (n+1)
12      t_retorn <- Right (aplicaSubst s1 (TFun t_s t1))
13      return (t_retorn, s1, n1)
14    )

```

Listing 2.5: Inferència de tipus per a abstraccions lambda i literals en línia

Per donar suport a cert nivell de reconeixement de patrons (*pattern matching*) directament als paràmetres (per exemple `\1 -> True`), s'utilitza una comprovació `all isDigit s`. Si el paràmetre és un numeral, l'interpret assigna directament el tipus fix `TInt`. Per contra, el cas general (quan s'empren lletres com `x` o `y`) es tracta enllaçant el nou identificador sintàctic amb una variable de tipus de treball fresca `TVar ("t"++ show n)`. Aquest supòsit s'injecta a l'inici del context abans de procedir a inferir el cos de l'expressió (`e`). Finalment, reconstrueix la definició pròpia de la funció mitjançant el constructor `TFun`.

Aplicació de Funcions

L'aplicació d'una funció a un argument (`App e1 e2`) és el punt on s'exigeix la màxima consistència estructural, ja que s'està instanciant efectivament el còmput:

```

1 infereix inctx ctx (App e1 e2) n = do
2   (t1, s1, n1) <- infereix inctx ctx e1 n
3   (t2, s2, n2) <- infereix inctx (aplicaSubstCtx s1 ctx) e2 n1
4
5   t3 <- Right (TVar ("t" ++ show n2))

```



```

6  s3 <- generaSubst (aplicaSubst s2 t1) (TFun t2 t3)
7
8  return (aplicaSubst s3 t3, s3 ++ s2 ++ s1, n2 + 1)

```

Listing 2.6: Inferència i unificació acumulativa de l'aplicació de funcions

El funcionament d'aquest bloc requereix quatre fases sincronitzades:

1. S'infereix el tipus del descriptor esquerre (**e1**), generant un tipus presumptament funcional **t1** i les seves respectives substitucions **s1**.
2. S'actualitza l'entorn de treball aplicant l'historial **s1** al context complet mitjançant **aplicaSubstCtx**, i a continuació s'infereix el tipus de l'argument dret (**e2**), obtenint **t2** i **s2**.
3. S'encunya una variable fresca completament aliena **t3** per representar la naturalesa de retorn encara no descoberta de l'aplicació subjacent.
4. Es propicia un escenari d'unificació: l'interpret força que el tipus esquerre (ja modelat prèviament i actualitzat amb el subconjunt **s2**) assumeixi formalment la identitat d'una funció **TFun t2 t3**. Posteriorment, el bloc retorna les capes de substitucions correctament combinades de dreta a esquerra (**s3 ++ s2 ++ s1**).

Estructures de Control i Definicions Recursives

L'anàlisi del constructe **If e1 e2 e3** exigeix una coherència de dades transversal, garantint que la condició sigui booleana i que ambdues branques convergeixin unívocament cap a la mateixa essència de dada, generant diversos **sAcc** (acumuladors lineals de substitucions) per garantir que la informació teòrica s'escampi per tota l'expressió seqüencialment.

Paral·lelament, l'algorisme contempla el repte fonamental de les declaracions pròpies amb referències auto-contingudes. La funció auxiliar **infereixDefRec** està dissenyada explícitament per tractar funcions recursives introduint una suposició anticipada (una variable **t** fresca) dins del context lèxic sota el nom de la pròpia funció, permetent referenciar-la dins del seu cos abans d'inferir completament quin tipus definitiu representa.

2.4 Representació i Estètica Formats (Pretty-Printing)

Més enllà del nucli de càlcul logicodeductiu intern, l'assoliment pràctic de tot aquest desplegament és lliurar al desenvolupador l'anàlisi semàntic de manera comprensible. Tot i que les variables formades numèricament (com `TVar "a1"`) proporcionen estabilitat algebraica per al processador HM, resulten redundants i confuses a l'hora de mostrar els resultats de tipus al bucle REPL de l'usuari [4].

Per suplir aquesta mancança visual, s'ha implementat un mòdul de transformació estètica a la funció `showTipus`:

```
1 showTipus :: Tipus -> String
2 showTipus TInt = "Int"
3 showTipus TBool = "Bool"
4 showTipus (TFun (TFun t1 t2) t3) = "(" ++ showTipus t1 ++ " -> " ++
    showTipus t2 ++ ")" ++ " -> " ++ showTipus t3
5 showTipus (TFun t1 t2) = showTipus t1 ++ " -> " ++ showTipus t2
6 showTipus (TVar a) = a
```

Listing 2.7: Funció de representació textual amb respecte a l'associativitat per la dreta

Donat que en la teoria de tipus funcional el constructor de funcions és estructuralment **associatiu per la dreta**, una expressió visualment modelada com a $A \rightarrow B \rightarrow C$ ja implica intrínsecament l'ordre agrupacional correcte $A \rightarrow (B \rightarrow C)$ sense necessitar parèntesis. Ara bé, si ens trobem en el cas oposat on el paràmetre que es rep per la banda esquerra és ell mateix una altra funció d'ordre superior com $(A \rightarrow B) \rightarrow C$, els delimitadors de lectura prenen una funció cabdal de la gramàtica.

La funció `showTipus` garanteix que la màquina discrimina de forma precisa quan un paràmetre esquerre posseeix forma funcional recursiva executant la branca `(TFun (TFun t1 t2) t3)`. Aquest factor aïlla la subestructura amb l'embolcall correcte per imprimir a la línia de comandes els tipus formals en el seu hàbit d'expressió canònica i fidel als canons imposats a l'ecosistema Haskell clàssic.

Capítol 3

Avaluació i Reducció de Grafs

La fase d'avaluació constitueix el nucli operatiu de l'interpret. Una vegada que el sistema ha verificat la correcció sintàctica i ha inferit satisfactòriament el tipus de l'expressió, l'arbre de sintaxi abstracta (AST) es transforma en una estructura de dades dinàmica capaç de suportar l'avaluació canònica mandrosa (*lazy evaluation*). En aquest capítol es detalla com es realitza la transició de l'AST cap a un graf dirigit, els mecanismes d'adreçament en memòria basats en mètodes d'estat pur, i l'estratègia de reducció per camí esquerre (*spine unwinding*) implementada en els mòduls `HSTDef.hs` i `HSEval.hs`, prenent com a base fonamental les teories d'implementació de llenguatges funcionals de Peyton Jones [5].

3.1 De l'AST al Graf

A diferència dels interprets ingenus basats en arbres purs, on cada subexpressió s'avalua de forma independent duplicant el treball computacional en cas de variables compartides, aquest interpret empra la tècnica de **reducció de grafs** (*graph reduction*) introduïda històricament per David Turner [7]. La diferència fonamental rau en el fet que un graf dirigit permet que múltiples nodes apuntin a una mateixa adreça de memòria. Això possibilita compartir subexpressions en lloc de copiar-les, establint la base física indispensable per a l'avaluació mandrosa eficient.

La transformació de l'AST cap al graf es troba governada per la funció `ast2graph` definida al mòdul `HSEval.hs`. Aquesta funció fa un recorregut descendent de l'AST i va construint seqüencialment els nodes corresponents dins del mapa de memòria global (*Heap*):

```
1 ast2graph :: Expr -> HeapState -> [(String, Addr)] -> (Addr, HeapState)
```

Listing 3.1: Funció de transformació de l'AST al graf elemental

La signatura rep l'expressió a processar (`Expr`), l'estat actual del monticle (`HeapState`) i un entorn d'adreces associatives (`[(String, Addr)]`) que mapeja els identificadors textuals de les variables cap a les seves posicions físiques existents en el graf. La funció retorna un parell format per l'adreça de l'arrel de la nova estructura generada (`Addr`) i l'estat actualitzat del monticle.

El funcionament intern es ramifica segons els constructors definits a `HSDef.hs`:

- **Valors literals** (`Val v`): Es genera un node atòmic `NVal v` i s'insereix en l'adreça lliure actual `a`, incrementant el comptador global per a futures allocacions.
- **Variables** (`Var s`): L'interpret efectua una cerca a la llista d'associacions `strAddr`. Si la variable ja existeix (és a dir, ha estat lligada per una lambda superior), es retorna la seva adreça original sense crear cap node nou, la qual cosa consolida formalment el **compartiment de referències** teoritzat per Turner [7]. Si no es troba, s'assumeix que és una variable global o lliure i s'introdueix un node `NVar s`.
- **Aplicacions** (`App e1 e2`): Es resol de manera recursiva la construcció de la branca esquerra (`e1`) i de la branca dreta (`e2`). Finalment, s'allotja un node connector `NApp a1 a2` que enllaça ambdues adreces ordenadament.
- **Abstraccions** (`Lam str e`): Es modela la unió sintàctica inserint un node `NLam` que apunta tant a la variable lligada (`NVar str` a la posició `a+1`) com al cos transformat (`a1`), forçant que durant la conversió del cos qualsevol aparició del token quedi associada inequívocament a aquesta posició.
- **Condicionals** (`If e1 e2 e3`): Es construeix un node de control combinat `NIf a1 a2 a3` que manté penjats els subgrafs de la condició, de la branca *then* i de la branca *else*.

3.1.1 Estructura del Heap i Adreçament

Per modelar el comportament del monticle de memòria conservant la puresa referencial del llenguatge amfitrió, s'ha definit un sistema d'adreçament lògic indexat a `HSDef.hs`:

```
1 type Addr = Int
2 type Heap = IM.IntMap Node
```

```
3 type HeapState = (Addr, Heap)
```

Listing 3.2: Definicions de l'estructura del monticle a HSDef.hs

L'ús de `Data.IntMap` (un mapa on les claus són exclusivament de tipus `Int`) com a base per representar el graf aporta avantatges arquitectònics molt significatius en comparació amb estructures d'arbres balançats genèrics o llistes de cerca lineals:

1. **Rendiment quasi constant ($O(\min(n, W))$):** Internament, `IntMap` s'implementa com un arbre de prefixos de Patricia de tipus Big-Endian. Això significa que les operacions de cerca (`lookup`), inserció (`insert`) i eliminació (`delete`) no depenen de forma directa del nombre total de nodes n , sinó del nombre de bits W de la clau (que en arquitectures de 64 bits és un màxim fix de 64). A la pràctica, per a la mida habitual dels grafs executats en l'interpret, aquest comportament és asimptòticament indistingible d'un rendiment d'ordre constant $O(1)$.
2. **Adreçament explícit sense punters reals:** Atès que Haskell és un llenguatge d'alt nivell amb recollidor de brossa propi, no és possible manipular adreces físiques de memòria RAM de forma directa. L'ús de tipus de dades `Addr = Int` simula de manera robusta una memòria d'accés aleatori controlada per programari.
3. **Filat d'Estat Pur (*State-Threading*):** L'estat del graf es propaga de manera transparent mitjançant el parell `HeapState`. El primer element representa la següent adreça lògica lliure disponible (*next-free pointer*), actuant com un comptador incremental que assegura la unicitat de cada node allotjat, eliminant col·lisions d'adreçament sense requerir mutabilitat lateral [8].

3.2 Lambda Lifting i Supercombinadors

L'avaluació eficient de llenguatges funcionals mitjançant reducció de grafs requereix l'eliminació total de les abstraccions lambda niuades que contenen dependències locals o variables ocultes del context circumdant. El formalisme utilitzat per resoldre aquest inconvenient és el procés de **Lambda Lifting**, el qual transforma un programa amb definicions de funcions locals en un conjunt homogeni de funcions globals pures conegudes com a **Supercombinadors** [5].

A la base de codi de l'interpret (`HSDef.hs`), l'entorn global diferencia aquestes categories estructurals mitjançant el tipus algebraic `EnvObj`:

```
1 data EnvObj = PrimDef Primitive Int -- Primitiva nuclear i la seva
    aritat
```

```
| FuncDef Expr      -- Funció pròpia (Supercombinador)
) i AST
```

Listing 3.3: Representació d'objectes de l'entorn global

Un `FuncDef` emmagatzema l'expressió purificada d'un supercombinador. Quan el motor d'avaluació ensopega amb el nom d'aquesta funció global (un node `NVar s` registrat al `DefEnv`), utilitza la plantilla de l'AST desada per instanciar directament un subgraf fresc dins del monticle actiu usant `ast2graph`, substituint els paràmetres formals per les adreces reals exposades en la pila d'execució.

3.2.1 Extracció de variables lliures

Una variable es diu lliure respecte a una abstracció lambda si apareix dins del cos de la funció però no està lligada pel paràmetre d'aquesta lambda ni per cap entorn global. Per exemple, a l'expressió $\lambda x.(\lambda y. + x y)$, la variable x és lliure dins de l'abstracció interna que defineix el paràmetre y .

L'extracció de variables lliures és el primer pas del *Lambda Lifting*. Consisteix a analitzar recursivament l'arbre sintàctic per calcular el conjunt de tokens lliures d'una subexpressió. Un cop identificades, aquestes variables s'afegeixen com a paràmetres explícits i formals de la funció interna. En l'exemple anterior, l'abstracció interna es reescriu afegint x com un argument addicional, transformant-se en una estructura equivalent a $(\lambda x.(f_{\text{interna}} x))$.

3.2.2 Generació de funcions globals

Un cop s'han extret totes les variables lliures d'una funció interna i s'han convertit en paràmetres, la lambda esdevé un **combinador pur** (manca completament de vincles externs). En aquest punt, es produeix la generació de la funció global: l'abstracció s'extreu completament de la seva posició local dins de l'AST i es promociona com una definició independent al catàleg de primer nivell del sistema (`preludeDefEnv` o equivalents).

Aquest procés garanteix que el motor d'avaluació del graf només hagi de gestionar definicions globals planes de tipus `FuncDef`. Quan s'invoca el supercombinador des del codi, l'interpret simplement aplica reduccions posicionals directes sobre la pila de termes, eliminant la necessitat de mantenir tancaments lògics dinàmics clàssics (*closures*) en temps d'execució i optimitzant dràsticament l'ús de la memòria del **Heap**.

3.3 Estratègia de Reducció (Lazy Evaluation)

L'estratègia d'avaluació de l'interpret segueix un model d'**avaluació mandrosa** o *lazy evaluation*, formalitzada teòricament mitjançant la reducció d'ordre normal [5]. Sota aquest model, les expressions no s'avaluen en el moment de ser passades com a arguments d'una funció, sinó únicament i exclusiva quan el seu valor numèric o lògic és estrictament demandat per una operació primitiva primària.

L'estat d'estabilitat d'un node es determina mitjançant el concepte de **Forma Normal de Cap Feble** o **WHNF** (*Weak Head Normal Form*). La funció `isWHNF` codifica aquesta condició de parada clau del sistema:

```
1 isWHNF :: (Addr, HeapState) -> Bool
2 isWHNF (a, (aa, hp)) =
3     let Just n = IM.lookup a hp
4     in case n of
5         NVal _      -> True
6         NVar _      -> True
7         NLam _ _    -> True
8         NApp _ _    -> (
9             let (b1, nApps, nPars) = getNumAppOfPredFunc (a, (aa, hp)) 0
10            in case b1 of
11                True  -> if nApps >= nPars then False else True
12                False -> False
13        )
14     NIf _ _ _ -> False
```

Listing 3.4: Funció de control de la Forma Normal de Cap Feble

Un node es considera estabilitzat en WHNF si és un literal numèric (`NVal`), una variable lliure sense codi associat (`NVar`), o una abstracció lambda (`NLam`), ja que el motor no avalua mai el cos intern d'una lambda fins que aquesta no rebi el seu corresponent argument. En el cas dels nodes d'aplicació (`NApp`), la funció `getNumAppOfPredFunc` interroga la branca esquerra per verificar si ens trobem davant d'una primitiva predefinida. Si el nombre d'aplicacions detectades a la pila (`nApps`) és inferior a l'aritat requerida per la primitiva (`nPars`), l'expressió representa una **aplicació parcial** i s'identifica legítimament com a WHNF. En cas contrari, o si es tracta d'un condicional `NIf`, el bloc requereix més reduccions i retorna `False`.

3.3.1 Exploració del Camí Esquerre (Spine Unwinding)

El corrent de control de l'avaluació incremental s'executa a la funció `pas`, la qual rep l'adreça arrel d'estudi, l'estat del monticle, l'entorn de definicions i una llista d'adreces que actua com a **pila de la línia dorsal** o **Spine Stack** (`[Addr]`).

Quan el sistema s'enfronta a una expressió d'aplicació (`NApp a1 a2`), s'activa el mecanisme conegut com a **Spine Unwinding** (exploració del camí esquerre). Donat que les aplicacions de funcions són associatives cap a l'esquerra [5], la funció real es troba amagada a l'extrem inferior esquerre de la cadena de nodes. El sistema realitza un descens recursiu acumulant l'adreça de cada node d'aplicació directament a la pila estructural:

```
1 NApp a1 a2 -> pas (a1, (aa, hp)) defEnv (a:stk)
```

Listing 3.5: Cas de l'Spine Unwinding a la funció `pas`

Aquest descens continua de forma ininterrompuda fins a trobar un node que no sigui una aplicació (normalment una lambda `NLam` o una variable de primitiva `NVar`). En aquest precís instant, la pila (`stk`) conté, ordenades de dalt a baix, les adreces dels nodes `NApp` que capturen els arguments de la funció.

Un cop assolit el fons esquerre, es bifurca la reducció segons el node descobert:

- **Beta-reducció** (`NLam a1 a2`): Es recupera el node d'aplicació superior de la pila (`app1`), el qual conté l'argument real a la seva branca dreta (`a3`). S'extreu el contingut d'aquest argument i s'enllaça directament substituint la variable formal `a1`. Posteriorment, es realitza un pas crucial d'actualització: el node d'aplicació `app1` es **reescriu completament en memòria** col·locant-hi el contingut del cos de la lambda (`brancaDretaLam`). Això garanteix que qualsevol altra part del graf que compartís aquest node d'aplicació visualitzi de forma instantània el resultat de la reducció, evitant duplicació de càlculs posteriors i consolidant el comportament mandrós.
- **Avaluació de Primitives** (`NVar s apuntant a un PrimDef`): S'extreuen de la pila els nodes d'aplicació necessaris segons l'aritat de l'operador. Atès que les primitives d'aquest intèrpret (com `Plus` o `Mult`) són **estrictes**, es requereix forçar l'avaluació prèvia dels seus arguments abans de realitzar el càlcul matemàtic. La funció comprova si els fills estan en WHNF; si no ho estan, crida recursivament a `pas` focalitzant-se en la posició de l'argument desprotegit. Un cop ambdós operands han col·lapsat cap a nodes `NVal`, s'invoca la funció de càlcul aritmètic, es converteix el resultat de nou a subgraf i s'actualitza el node d'aplicació arrel de l'operació per contenir el valor final.

Aquest disseny d'actualitzacions *in situ* sobre l'`IntMap` d'estat pur garanteix l'harmonia absoluta entre la immutabilitat lògica del llenguatge de programació Haskell i l'eficiència pràctica d'una màquina de reducció de grafs.

3.3.2 Cas Pràctic: Avaluació Pas a Pas de l'Expressió Guia

Per il·lustrar de manera didàctica l'aplicació d'aquests mecanismes, s'analitza a continuació l'execució de la nostra expressió guia: `(\x y -> x * y) 2 3`. Al mòdul `HSEval.hs`, la funció `hsprint` està programada per traduir l'estat del `HeapState` intern a una cadena de text llegible a cada pas del bucle d'avaluació, oferint a l'usuari una visió transparent del motor de reducció.

La consola de l'interpret mostra la següent traça d'estats successius:

```
1 bhct> (\x y -> x * y) 2 3
2 (\x y -> (*) x y) 2 3
3 (\y -> (*) 2 y) 3
4 (*) 2 3
5 6
```

Listing 3.6: Traça d'avaluació de la currificació i reducció de l'expressió guia

Aquesta seqüència reflecteix fidelment el recorregut teòric presentat:

1. **Instanciació Inicial:** La cadena original `(\x y -> x * y) 2 3` passa per l'AST i es bolca al graf. S'observa com l'operador infix `*` ha estat reconegut com una aplicació predefinida associada a l'esquerra: `(*) x y`.
2. **Primera Beta-reducció (*Spine Unwinding*):** L'interpret descendeix pel camí esquerre dels nodes d'aplicació fins arribar a la lambda exterior `\x`. Al cim de la pila d'arguments (`stk`) hi troba el valor 2. En aquest pas, substitueix de manera estructural totes les instàncies del node lligat a la `x` pel node literal 2. El node d'aplicació arrel d'aquesta unió es reescriu amb el nou cos generat. El graf intern ara reflecteix la clàusula currificada parcialment avaluada: `(\y -> (*) 2 y) 3`.
3. **Segona Beta-reducció:** El mecanisme de l'espina esquerra es repeteix per a l'aplicació restant. Ara es descendeix fins a `\y` i se li assigna el valor que espera al cim de la pila, un 3. La nova reescriptura del graf col·lapsa l'abstracció i deixa aïllada l'expressió `(*) 2 3`.
4. **Execució Estricta de la Primitiva:** S'assoleix el node corresponent a `(*)`, el qual està lligat a una primitiva aritmètica `Mult` amb aritat 2 dins del `preludeDefEnv`.

Com que l'operador demana l'avaluació estricta, s'assegura que els arguments (adreces de memòria dels nodes 2 i 3) ja es trobin en *WHNF* (com són *NVal*, ho estan). S'executa l'operació numèrica a la CPU subjacent i el graf se sobreescriu amb l'únic node atòmic resultant *NVal* 6. Com que ja no queden més aplicacions a resoldre i el graf compleix la condició total d'estabilitat (*isWHNF*), l'interpret atura el cicle i imprimeix el resultat final: 6.

Capítol 4

Resultats i Proves

En aquest capítol es descriu l'estratègia de validació utilitzada per comprovar la correcció tant de l'analitzador sintàctic (AST) com dels mòduls d'inferència de tipus i d'execució de supercombinadors. Per fer-ho, s'ha dissenyat una bateria de tests estructurada en fitxers independents que exploren els límits del llenguatge.

4.1 Bateria de Tests d'Inferència

La validació del sistema de tipus s'ha dividit en dues fases clarament diferenciades mitjançant els fitxers `test_ast.hs` i `test_type.hs`: la comprovació de la correcta generació de l'arbre de sintaxi abstracta (AST) i la resolució de tipus mitjançant l'algorisme de Hindley-Milner.

4.1.1 Verificació de l'AST (`test_ast.hs`)

Aquesta primera línia de proves utilitza la comanda `:a` per assegurar que les expressions constants, operacions aritmètiques, abstraccions lambda i aplicacions parcials es parsegen creant l'estructura de nodes esperada. S'inclouen expressions com:

- **Primitives i Operacions:** `123`, `12 + 1` i constants booleanes (`True`).
- **Lambdes i Ordre Superior:** Abstraccions simples (`\x -> x + 1`), multi-argument (`\x y -> x + y`) i la composició de funcions (`(.)` `(\x -> x*3)`).

4.1.2 Inferència Exitosa i Polimorfisme (`test_type.hs`)

Mitjançant la comanda `:t`, es posa a prova la capacitat de l'inferidor per gestionar el polimorfisme paramètric (*Let-Polymorphism*) i les funcions d'ordre superior:

- **Instanciació múltiple:** Es valida que funcions com `id` o `const` es puguin usar amb diferents tipus en un mateix entorn (`id True` en front de `id id`) sense unificar erròniament les seves variables de tipus genèriques.
- **Composició de funcions:** Es resol el tipus de l'operador composició `(.)` combinat amb funcions booleanes (`not`) i aritmètiques.

4.1.3 Tests Negatius i Robustesa

Per garantir que l'interpret no es deté abruptament davant de codi erroni, s'han inclòs casos de prova dissenyats per fallar i llançar excepcions controlades:

- **Variables no lligades:** L'ús de `variableFantasma` o variables lliures en lambdes (`\x -> x + y`) és capturat correctament per l'entorn de noms.
- **Test d'Occurrència (*Occurs Check*):** Expressions clàssiques del Càlcul Lambda pur com `\x -> x x` forçarien un tipus infinit. El sistema ho detecta i evita el bucle infinit en la unificació llançant un error de tipus.
- **Incoherència de Tipus (*Type Mismatch*):** S'avaluen negativament operacions invàlides com `1 + True`, branques de condicionals amb tipus heterogenis (`if True then 1 else False`) o aplicacions de valors no funcionals (`True 5`).

4.2 Execució de Supercombinadors

Els fitxers de reflexió d'execució (`test_eval_0.hs` fins a `test_eval_11.hs`) validen el reductor de grafs, la generació d'entorns i, especialment, l'avaluació mandrosa (*lazy evaluation*).

`test_eval_0`, `10` i `11` (**Recursió**): Es demostra la capacitat del motor per gestionar bucles recursius complexos. Destaca `factorial` amb acumulador (recursió de cua) i `aplicaVegades`, que avalua funcions d'ordre superior de manera iterativa aplicant lambdes en temps d'execució.

test_eval_1 i 9 (*Shadowing*): Garanteixen que l'àmbit lèxic manté la integritat de les dades quan s'oculten variables. A **extrem**, la redefinició contínua del nom **x** es resol correctament avaluant de dins cap a fora.

test_eval_2, 3 i 7 (*Closures i Currificació*): Proven que les funcions parcials capturen correctament el seu entorn lèxic de variables. Per exemple, **creaMultiplicador 5** genera una cloure vàlida que manté el valor 5 guardat per a futures aplicacions.

test_eval_5 i 8 (*Laziness*): Són els tests crítics d'avaluació mandrosa. L'expressió **omega x = omega x** (o **bucle x**) defineix un bucle divergent no terminatiu. Com que **constant 99** (**omega 0**) i **mandra 0** no necessiten avaluar l'argument divergent per retornar el resultat, l'interpret finalitza correctament (**99** i **42** respectivament) en lloc de quedar-se congelat.

4.3 Limitacions actuals

Tot i la robustesa del nucli del llenguatge implementat, s'han delimitat certes fronteres tècniques per acotar l'abast del projecte i garantir la seva viabilitat en el temps establert:

- **Absència de Tipus Algebraics de Dades (ADTs):** El llenguatge només dona suport natiu a tipus primitius binaris i numèrics (**Int**, **Bool**). No s'ha implementat l'estructura de dades complexa com llistes (**[]**), tuples o tipus de dades personalitzats definits per l'usuari (**data**).
- **Manca de *Pattern Matching*:** Com a conseqüència directa de la limitació anterior, definicions de lambdes basades en patrons o valors constants de manera directa (com es mostra als tests **\1 -> True** o **\True -> 123**) no estan suportades per la sintaxi actual i generaran un error en l'anàlisi sintàctica o de tipus.
- **Recursivitat Mútua o Creuada:** L'algorisme de resolució de definicions recursives (**infereixDefRec**) està dissenyat per tractar declaracions individuals que es criden a si mateixes. El sistema actual no admet blocs de funcions que depenguin jeràrquicament les unes de les altres de manera circular (recursivitat encreuada).

Capítol 5

Conclusions

Aquest darrer capítol tanca la memòria del projecte, fent balanç de la feina desenvolupada al llarg de tots aquests mesos. Es presenta una avaluació crítica dels resultats obtinguts respecte a les metes inicials, una reflexió sobre el procés d'aprenentatge viscut durant la implementació, i finalment, es plantegen possibles vies d'ampliació per evolucionar el sistema en el futur.

5.1 Objectius assolits

L'objectiu principal d'aquest treball era dissenyar i implementar des de zero un intèrpret funcional completament operatiu, bastit exclusivament sobre Haskell i sense dependre d'eines externes de generació de codi. Després d'analitzar els resultats obtinguts i el funcionament del sistema, es pot afirmar que aquest objectiu s'ha assolit amb èxit.

De manera més específica, s'han complert les següents fites tècniques:

- **Construcció d'un parser natiu i robust:** S'ha implementat amb èxit una arquitectura d'anàlisi lèxica i sintàctica basada en combinadors monàdics. Aquest enfocament ha permès analitzar amb èxit expressions complexes, respectant la precedència d'operadors i l'associativitat per l'esquerra pròpia de l'aplicació de funcions.
- **Motor d'inferència de tipus segur:** S'ha integrat un algorisme d'inferència basat en el sistema Damas-Milner, capaç de deduir estructuralment els tipus de les expressions i de suportar polimorfisme paramètric, dotant el llenguatge d'una capa de seguretat estàtica prèvia a l'execució.

- **Avaluació mandrosa i gestió de memòria:** El nucli de l'interpret s'ha construït sobre un model d'avaluació mandrosa (*lazy evaluation*), on l'ús de *thunks* (promeses de càlcul) garanteix que les expressions només es redueixen quan són estrictament necessàries. Això ha permès tractar de forma natural avaluacions parcials i currificació.
- **Disseny modular i homogeni:** Tot el codi manté una arquitectura purament funcional i declarativa, on les fases de *parsing*, tipificació i avaluació estan clarament desacoblades, complint amb els estàndards de l'enginyeria de programari moderna.

5.2 Aprenentatges personals

Més enllà dels resultats tècnics o del codi produït, la realització d'aquest Treball de Final de Grau ha suposat un repte intel·lectual de primer ordre i m'ha permès assolir aprenentatges molt valuosos.

En primer lloc, he consolidat de manera profunda la meua comprensió del paradigma de la programació funcional. Passar de ser un simple usuari d'un llenguatge a haver de programar-ne el funcionament intern obliga a entendre d'arrel conceptes com el Càlcul Lambda, les clausures lèxiques (*closures*), l'aplicació parcial i els efectes controlats mitjançant mònades.

En segon lloc, m'ha permès tancar la bretxa entre la teoria i la pràctica de l'enginyeria de llenguatges. Llegir la teoria sobre arbres de sintaxi abstracta o regles d'inferència és molt diferent de la complexitat pràctica de programar la substitució dinàmica de variables i evitar les col·lisions d'entorns d'execució.

Finalment, a nivell metodològic i organitzatiu, he après a gestionar un projecte de programari de llarga durada. Això inclou no només la picada de codi i l'arquitectura de programari, sinó també la recerca bibliogràfica acadèmica, la lectura de *papers* fundacionals i la redacció formal en $\text{\LaTeX}$, eines que sens dubte seran fonamentals per a la meua futura carrera professional.

5.3 Treball futur

El disseny de llenguatges és un camp pràcticament inabastable, i per aquest motiu, qualsevol interpret acadèmic té marge de creixement. Tot i que el sistema actual és complet i

operatiu dins del seu abast teòric, es proposen les següents línies de treball per a futures iteracions o ampliacions del projecte:

- **Millora en la traçabilitat d'errors:** Actualment, quan el *parser* falla o hi ha un error de tipus genèric, els missatges retornats a l'usuari són funcionals però aspres. Una millora significativa consistiria a afegir un seguiment de posició (línia i columna) per emetre missatges d'error formatajats a l'estil dels compiladors moderns.
- **Tipus de dades algebraics (ADTs) i *Pattern Matching*:** L'extensió natural del llenguatge requeriria dotar a l'usuari de la capacitat de definir els seus propis tipus (com llistes o arbres enumerats) i introduir construccions sintàctiques de reconeixement de patrons tipus `case ... of`, seguint els estàndards actuals.
- **Compilació cap a una màquina virtual (STG):** Tot i que l'interpret actual camina per l'arbre (AST) de forma eficaç, per aconseguir rendiments propers a compiladors reals com GHC, el pas següent seria traduir l'AST a instruccions d'una *Spineless Tagless G-machine* (STG), abandonant la interpretació directa a favor d'un entorn de memòria altament optimitzat.
- **Llibreria estàndard (*Prelude*):** L'ampliació de l'entorn global per defecte amb una col·lecció substancial de funcions predefinides sobre llistes, cadenes de text i manipulació matemàtica abstracta dotaria de molta més expressivitat pràctica l'usuari final del llenguatge.

Bibliografia

- [1] Luis Damas i Robin Milner. ?Principal type-schemes for functional programs? A: *Proceedings of the 9th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*. 1982, pàg. 207 - 212.
- [2] FIB. *Mòdul 2: Gestió de Projectes*. Apunts de l'assignatura GEP. 2026.
- [3] Graham Hutton i Erik Meijer. *Monadic Parser Combinators*. Inf. tèc. NOTTCS-TR-96-4. Nottingham, UK: Department of Computer Science, University of Nottingham, 1996.
- [4] Graham Hutton i Erik Meijer. ?Monadic parsing in Haskell? A: *Journal of Functional Programming* 8.4 (1998), pàg. 437 - 444.
- [5] Simon L. Peyton Jones. *The Implementation of Functional Programming Languages*. Prentice Hall, 1987. ISBN: 0-13-453333-X.
- [6] Daan Leijen i Erik Meijer. *Parsec: Direct style monadic parser combinators for the real world*. Inf. tèc. UU-CS-2001-27. Utrecht University, 2001.
- [7] David A. Turner. ?A new implementation technique for applicative languages? A: *Software: Practice and Experience* 9.1 (1979), pàg. 31 - 49.
- [8] Philip Wadler. ?The essence of functional programming? A: *Proceedings of the 19th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*. 1992, pàg. 1 - 14.